

Silent-App-Aware Federated Machine Unlearning for Encrypted Network Traffic Classification

Zeyi Li , Graduate Student Member, IEEE, Yuna Jiang , Member, IEEE, Tianshun Wang , Member, IEEE, Pan Wang , Member, IEEE, and Yimu Ji 

Abstract—In recent years, federated learning-based network traffic classification has emerged as a promising Artificial Intelligence (AI) approach for encrypted traffic identification, enabling accurate classification while preserving data privacy. To meet the rising demand for the right to be forgotten, Federated Machine Unlearning (FMU) has been introduced as an AI-driven forgetting paradigm. However, conventional FMU methods overlook silent applications removed due to guideline violations, resulting in reduced classifier stability and limited aggregation scalability. To address these challenges, this paper proposes FedUnSilApp, short for FMU for silent applications, a novel FMU framework that integrates a learning module, a forgetting module, and a scalable aggregation strategy. In particular, the forgetting module prunes neurons associated with silent applications in the classifier, minimizing their residual influence. The scalable aggregation strategy separately processes discriminative layers of clients requesting forgetting and those of other clients, thereby enhancing both stability and scalability of the global model. In addition, FedUnSilApp is applied to the real-world scenario of encrypted network traffic classification, enabling compliance with deletion requests while maintaining robust classification performance. The effectiveness of FedUnSilApp is validated through experiments, which show that FedUnSilApp outperforms state-of-the-art methods, improving accuracy by over 6% on the NJUPT dataset.

Index Terms—Machine unlearning, federated learning, federated machine unlearning, network traffic classification, encrypted network traffic.

Received 18 December 2025; revised 27 January 2026; accepted 3 March 2026. Date of publication 10 March 2026; date of current version 24 March 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62401283, in part by the State Key Laboratory for Novel Software Technology at Nanjing University under Grant KFKT2025B69, in part by the Suzhou High Performance Programmable Switching Chip Innovation Consortium under Grant LHT202326, and in part by the Development of an Ultra-large-scale Ubiquitous Network Quality Monitoring System Based on Trusted Edge Intelligence under Grant SYG202311. Recommended for acceptance by Dr. Yuan Wu. (Corresponding author: Pan Wang.)

Zeyi Li is with the School of Computer Science, Nanjing University of Posts and Telecommunications, Nanjing 210003, China (e-mail: 2022040506@njupt.edu.cn).

Yuna Jiang and Tianshun Wang are with the School of Communications and Information Engineering, Nanjing University of Posts and Telecommunications, Nanjing 210003, China (e-mail: yunajiang@njupt.edu.cn; tswang@njupt.edu.cn).

Pan Wang is with the School of Modern Posts, Nanjing University of Posts and Telecommunications, Nanjing 210003, China (e-mail: wangpan@njupt.edu.cn).

Yimu Ji is with the School of Computer Science, Nanjing University of Posts and Telecommunications, Nanjing 210003, China, also with the Institute of High Performance Computing and Bigdata, Nanjing University of Posts and Telecommunications, Nanjing 210003, China, and also with Jiangsu HPC and Intelligent Processing Engineer Research Center, Nanjing 210003, China (e-mail: jiyim@njupt.edu.cn).

Data and Code: <https://github.com/sailorlee97/FedUnSilApps>
Digital Object Identifier 10.1109/TNSE.2026.3672152

I. INTRODUCTION

AS AN important tool for network management and security, Network Traffic Classification (NTC) has received significant attention from both academia and industry since the late 1990 s. It has been widely applied in areas such as QoS/QoE management, network resource optimization, congestion control, and intrusion detection [1]. With the rapid development of next-generation communication technologies such as 5G/6G [2], [3], network technologies are evolving towards a highly autonomous direction of ‘zero touch’ [4], [5], [6]. As one of the decision-making tools for network services and security management, NTC plays a crucial role [7]. However, the rise of HTTPS encryption has rendered traditional Deep Packet Inspection (DPI) [8] methods ineffective for capturing user behavior patterns [9]. In response, Deep Learning (DL)-based NTC has emerged as a promising solution [10], [11], [12], as it can accurately identify user preferences without being restricted by encryption, drawing increasing attention from researchers.

Traditional DL-based NTC methods usually collect traffic interaction data from multiple edge devices. This data is aggregated to train a unified model in the cloud, which is subsequently deployed back to each edge device for inference [13]. However, as data privacy concerns continue to intensify, centralized machine learning approaches that require access to all training data are becoming increasingly infeasible [14]. To address these issues, Federated Learning (FL) is envisioned as a promising approach with enhanced privacy [15], [16], [17], [18], [19]. Specifically, FL allows model training without exposing user data, effectively alleviating privacy leakage issues for participants, and thus has become a focus of research in the field of NTC [20].

In recent years, with the rapid development of AI technology [21], the rights of the data subject¹ have also garnered increasing attention from society. The General Data Protection Regulation (GDPR) of the European Union [22] and the California Consumer Privacy Act (CCPA) [23] have introduced relevant provisions that grant participants in FL the ‘right to be forgotten’, meaning that participants have the right to request companies to delete their personal data and contributions. Since traditional FL-based NTC cannot ensure that participants can withdraw their data contributions [24], [25], researchers have begun to focus on Federated Machine Unlearning (FMU).

Currently, FMU methods can be categorized into three main approaches. One common FMU approach is retraining or fine-tuning the model with remaining data [26], [27]. Another method uses gradient ascent to reduce the influence of forgotten

¹ <https://gdpr-info.eu/art-17-gdpr/>

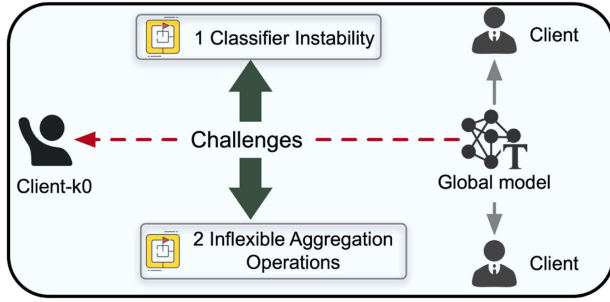


Fig. 1. $Client_{k_0}$ issues a forgetting request facing two obstacles: (i) classifier instability and (ii) inflexible aggregation operations.

data [28], while a third approach involves pruning neurons closely linked to the data to be forgotten [29]. However, these methods overlook two important requirements in the field of NTC: *stability* and *scalability*. Particularly, stability refers to the ability to maintain the model's original performance after unlearning, and scalability refers to the framework's ability to quickly adapt to both application forgetting and application re-activation. The corresponding challenges are as follows (see Fig. 1):

- 1) *Classifier Instability under Silent Applications*: Google removes approximately 25,100 applications from the Play Store every quarter, and these applications naturally become silent applications in NTC scenarios [30]. When silent applications remain in the classifier space, models tend to misclassify traffic belonging to active classes into silent ones, leading to degraded stability.
- 2) *Inflexible Aggregation in FL*: Unlike classical FL aggregation where the entire model is uniformly averaged, regulatory differences cause some clients to discard certain applications while others continue to classify them. When these applications are re-activated, the model is required to quickly regain its classification capability. Existing FL-NTC methods lack scalable aggregation mechanisms, making this adaptation slow and unstable.

Therefore, a natural question is: *could the framework achieve stability and scalability at the same time?* To address the challenges outlined above, this paper proposes a scalable class discriminative pruning-based FMU framework for NTC, called FedUnSilApp. The framework allows specific clients to request the removal of contributions from certain data categories, thereby achieving selective data forgetting. FedUnSilApp also processes these requests by isolating and aggregating the discriminative layer parameters of clients that require forgetting separately from those of other clients. This selective aggregation dynamically updates the global model, ensuring that it adapts to changing data requirements without compromising performance. This paper systematically addresses these challenges and makes the following contributions:

- 1) We point out this problem related to the stability and scalability of FL-based NTC caused by silent applications, which has been overlooked by previous researchers. To address this, we propose a framework called FedUnSilApp for NTC, which effectively mitigates the impact of silent applications.
- 2) To tackle the stability challenge, a forgetting mechanism is developed that freezes the feature extractor and prunes silent-app neurons in the classifier, ensuring stable predictions when application categories disappear.

- 3) To tackle the scalability challenge, a scalable aggregation strategy is proposed, whereby the feature extractor is aggregated across all clients to retain global generalisation. Meanwhile, classifier layers are aggregated separately for clients that forget and those that do not, enabling fast recovery when silent applications become active again.
- 4) Experiments on four datasets demonstrate substantial performance gains, with FedUnSilApp reaching 88.16% accuracy on NJUPT, outperforming state-of-the-art baselines (74.17%–81.7%) while maintaining efficiency.

The paper is structured as follows: Section II reviews the evolution and progression of NTC methodologies. Section III introduces the necessary preliminaries for the proposed method. Section IV provides a comprehensive description of the FedUnSilApp framework. Section V presents an overview of the experiments and evaluations, along with a discussion of the results. Finally, Section VI concludes the paper and outlines directions for future work.

II. RELATED WORKS

Encrypted traffic has rendered traditional NTC methods ineffective. In particular, with the advent of TLS 1.2 and TLS 1.3, these protocols establish security parameters and negotiate cryptographic keys during the Handshake stage, and employ symmetric encryption to protect application data during the Record Layer stage. Consequently, traditional classification approaches based on port features, header features, and payload features have become largely obsolete. However, the emergence of AI has brought new perspectives and opportunities for NTC. As illustrated in Fig. 2, this section presents the three major evolutionary waves of AI-driven NTC: Machine Learning (ML)-based, DL-based, and FL-based approaches.

A. ML-Based Ntc

With the rise of ML, researchers began to move beyond traditional port-based, header-based, and payload-based traffic classification and instead learn discriminative patterns directly from statistical features of flows. Early work showed that supervised ML algorithms can effectively classify application traffic without relying on well-known ports or payload inspection, which are increasingly unreliable due to encryption and dynamic port usage [31].

Subsequent studies applied ML methods to more challenging encrypted traffic environments. The research [32] has shown that, despite traffic encryption, dynamic port changes, and invisibility of payloads, classical supervised learning models can still effectively perform classification tasks. The work [33] proposes a second-order Markov chain-based encrypted traffic classifier enhanced with application attribute bigrams to improve discrimination among applications with similar TLS fingerprints. The research [34] applies multinomial Naïve Bayes to normalized packet-size frequency distributions for system-agnostic website fingerprinting, achieving very high identification accuracy. The work [35] proposes a constrained clustering approach that leverages TCP/IP-inferred equivalence constraints to enhance the accuracy and convergence of unsupervised Internet traffic classification.

ML-based NTC serves as a transitional paradigm between rule-based detection and AI-driven approaches. While it reduces reliance on manual protocol signatures and improves adaptability to encrypted traffic, it still depends heavily on handcrafted

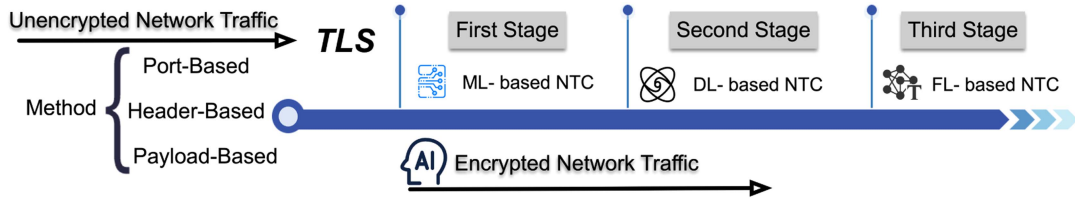


Fig. 2. Evolution of NTC methods from unencrypted to encrypted traffic, illustrating the transition from port/header/payload-based approaches to ML-, DL-, and FL-based NTC under increasing encryption (TLS).

features, limiting scalability and representation capability. Consequently, ML-based methods are increasingly supplanted by DL approaches that offer more effective automatic feature learning for large-scale and complex traffic data.

B. DL-Based Ntc

After the initial exploration of ML-based approaches, DL-based methods for NTC have gradually emerged [36], [37]. Particularly, when dealing with encrypted traffic, complex data structures, and highly temporal flow patterns, DL has demonstrated its distinctive advantages in automatic feature extraction and robust representation learning [38], [39].

Among the representative studies, TRCLA [40] introduces a transfer-learning algorithm based on Cellular Learning Automata (CLA) that employs two novel decision criteria—merit and attitude parameters—to alleviate negative transfer and improve classification accuracy in transductive settings. ABLTC [41] proposes an attention-based LSTM architecture with lightweight preprocessing to balance efficiency and accuracy, while Yao et al. [42] enhance recurrent neural networks with attention-aided LSTM and Hierarchical Attention Network (HAN) modules to capture temporal dependencies and achieve higher accuracy on the ISCX VPN–NonVPN dataset. To further address data imbalance and generalization, Zheng et al. [43] present the Flow-Based Relation Network (RBRN), an end-to-end meta-learning framework that learns flow-level representations directly from raw data and introduces a hallucinator module to generate balanced training samples.

A parallel research line explores CNN-based end-to-end modeling. FlowPic [44] transforms raw flow data into image-like representations and applies Convolutional Neural Networks (CNNs) for automatic classification, achieving up to 99.7% accuracy even on VPN and Tor traffic. Wang et al. [45] further propose an end-to-end 1D-CNN model that unifies feature extraction and classification, while deep packet [46] combines stacked autoencoders and CNNs into a single architecture, achieving state-of-the-art performance on VPN and non-VPN traffic. Dong et al. [47] present CETAnalytics, a neural network-based framework that jointly analyzes payload content and statistical features to attain both high precision and strong generalization. For encrypted protocols, Tong et al. [48] design a CNN-based QUIC traffic classifier that integrates feature extraction and classification to reach a 99.24% F1-score on Google services.

In addition, the work [49] employs a multi-layer encoder-decoder recurrent architecture to deeply mine sequential dependencies of encrypted flows and enhance feature representation through a reconstruction mechanism, achieving 99.14% TPR and outperforming prior models. Al-Obaidy et al. [50] focus on social-media encrypted traffic, demonstrating that ML-based

methods can effectively identify sub-classes of popular applications such as Skype, WhatsApp, Facebook, and YouTube. For online and interpretable learning, the research [51] introduces a packet-level self-attention network enabling real-time classification and interpretability by visualizing attention weights. To improve scalability in large-scale multi-class settings, the study [52] proposes a hierarchical recurrent neural network that decomposes classification tasks into smaller subtasks using a tree-structured multi-classifier design, improving both accuracy and convergence speed.

Beyond performance improvements, several works examine robustness and generalization. Sadeghzadeh et al. [53] introduce Adversarial Network Traffic (ANT), a Universal Adversarial Perturbation (UAP)-based framework for evaluating the resilience of DL-based NTC models against crafted perturbations. Collectively, these studies reflect the evolution of DL-based NTC from early CNN/RNN-driven architectures toward more advanced paradigms incorporating attention mechanisms, meta-learning strategies, hierarchical modeling, and adversarial robustness, marking a significant milestone in the second wave of AI-driven traffic classification and paving the way for the upcoming FL-based NTC era.

C. FL-Based Ntc

With the increasing emphasis on data privacy and security, traditional centralized model training approaches in NTC have faced growing challenges such as data silos and privacy leakage [54]. To address these issues, FL has emerged as a promising paradigm that enables collaborative model training without requiring the exchange of raw data [55], [56], [57]. In an FL-based framework, participating entities locally train models on their respective datasets and share only model parameters or gradients, thereby preserving user privacy while enabling cross-domain collaboration [58], [59].

This decentralized learning strategy not only mitigates privacy risks but also leverages diverse and geographically distributed traffic data to improve generalization and robustness. FL-based NTC represents the third technological wave following deep learning, marking a new stage in the evolution of AI-driven network intelligence and security research [60].

Recent studies have explored various FL architectures and optimization strategies for NTC. The study [61] proposes an Automated Separate Guided Attention Federated Graph Neural Network, which integrates a hybrid Vision Transformer with bidirectional LSTM for multi-scale feature extraction and employs federated graph learning for distributed intrusion detection. The model achieves over 99% accuracy across multiple cybersecurity datasets, demonstrating the efficiency of attention-enhanced federated architectures. Similarly, the research [62]

introduces a federated analytics framework to mitigate data heterogeneity and non-IID distributions. To further enhance model reliability, the work [63] proposes WCL, a client selection strategy that combines model weight divergence with local training loss to achieve balanced aggregation and faster convergence. Experiments on QUIC and ISCX datasets verify that WCL outperforms conventional approaches under both low and high heterogeneity. In the IoT domain, He et al. [64] introduce FedeEDI, a federated edge device identification framework leveraging network traffic features for IoT security. Compared with centralized learning, FedeEDI provides faster training, decentralized deployment, and stronger data protection.

Beyond classification, FL has also been extended to network optimization tasks. Huang et al. [65] propose a QoS-aware federated caching framework in fog-enabled IoT networks, incorporating a distributed cluster-based preference estimation algorithm to mitigate non-IID data effects and improve cache hit rate, convergence, and learning stability. Similarly, Jiang et al. [66] design FedSL-LSTM, a federated split learning framework for sequential data analysis in satellite-terrestrial integrated networks. By combining FL, split learning, and LSTM-based modeling, FedSL-LSTM achieves competitive accuracy while maintaining communication efficiency and privacy preservation.

Furthermore, other recent works have extended the use of FL for anomaly detection and privacy-sensitive traffic analysis. Studies such as [67] address the challenge of detecting anomalies in multi-time series data across distributed network devices, emphasizing the complexity of aggregating non-IID data while maintaining strict privacy guarantees. Likewise, Bakopoulou et al. [20] highlight the challenges of training traffic classifiers on mobile packet data containing sensitive information such as personally identifiable information and advertising metadata, proposing privacy-preserving FL-based solutions.

In a related contribution, Zhang et al. [68] propose an adaptive label normalization mechanism that transforms noisy labels to better align with clean data distributions, improving robustness and accuracy in FL settings. Finally, Pekar et al. [69] explore incremental federated training, where the model evolves continuously over successive rounds by incorporating new data from participating clients. This incremental strategy enhances model adaptability and efficiency in dynamic environments, demonstrating the growing maturity of FL as a viable solution for privacy-preserving and continuously learning NTC systems.

D. Federated Machine Unlearning

With the increasing emphasis on the ‘right to be forgotten’ [74] within FL, FMU has emerged as a natural extension of privacy-preserving collaborative learning [75]. While traditional FL primarily focuses on preventing data leakage through decentralized training, it does not inherently provide mechanisms to remove previously learned information once a client or data sample requests deletion. Addressing this limitation, FMU aims to enable selective, efficient, and verifiable removal of data contributions from global models—thereby advancing FL from passive privacy protection to active data controllability. In this context, FMU is driving a new wave of innovation in FL ecosystems, fostering a new research frontier that bridges data privacy, model accountability, and compliance with global data regulations.

This study [76] is the first to apply unlearning in network traffic data. It presents a novel machine unlearning framework

called ViFLa to address the efficiency and adaptability required in IoT traffic anomaly detection. ViFLa introduces the concept of virtual FL, where training data is grouped based on their likelihood of needing to be unlearned. Additionally, scholars have conducted extensive research on FMU [77]. The research [72] focuses on overcoming the limitations of existing FMU methods, which often require time-consuming re-training and lack the ability to handle diverse unlearning tasks. FedAU incorporates a lightweight auxiliary unlearning module that allows for quick and targeted unlearning through a simple linear operation. This method supports unlearning at multiple levels—individual samples, specific classes, or entire clients—providing broad unlearning capabilities. The research [29] focuses on the class-level ‘forgetting’ problem in FL scenarios and proposes a method that enables selective information removal for specific classes in a trained CNN classification model. This approach achieves the selective information removal without direct access to global training data and without requiring complete retraining. Utilizing TF-IDF-guided channel pruning techniques, this method accomplishes an efficient and accurate ‘forgetting’ process. Experimental results demonstrate significant advantages in improving forgetting speed and maintaining model accuracy. The study [73] introduces a new FedRecovery algorithm that employs differential privacy techniques to implement unlearning tasks in FL frameworks. It can eliminate the influence of client data without retraining the model. This method is especially suitable for complex models and dynamic FL environments. Although it provides strong privacy protection and theoretical guarantees, it may still face challenges such as performance impact from noise and computational overhead in certain applications. The authors in [28] proposed and evaluated two data deletion methods for deep neural networks, namely, Unlearning and Amnesiac Unlearning, to meet the GDPR’s ‘right to be forgotten’ compliance requirements. These methods efficiently remove sensitive data while preserving model performance, effectively defending against model inversion and membership inference attacks. Despite limitations in applicability, sample-level deletion, model complexity, and rapid response, they offer important references for the development of data privacy protection technologies. The work [78] presents an efficient FMU algorithm called FFMU, which combines nonlinear functional analysis and the Nemytskii operator. This approach not only enhances FMU efficiency but also preserves model performance during the data deletion process, meeting FL’s privacy protection requirements. The FFMU method avoids redundant training steps found in traditional FMU approaches and ensures efficient collaboration between local and global models. However, sample-level ‘forgetting’ and applications in more complex models remain important topics for future research. The research [79] explores methods to remove backdoor attacks in FL and proposes an effective federated unlearning method. This approach eliminates the impact of identified attackers by subtracting their historical parameter updates from the global model and utilizes knowledge distillation to restore model performance, ensuring that, once attackers are identified, their backdoor contributions are thoroughly removed while preserving model accuracy and efficiency. The study [80] offers a new solution for defending against backdoor attacks in FL by removing historical updates from attackers and combining knowledge distillation to effectively restore model performance, avoiding reliance on clients. It has high computational efficiency and low energy consumption, making it highly promising for real-world applications. However, the method still encounters

TABLE I
COMPARISON OF REPRESENTATIVE RESEARCH WORKS ACROSS ML-BASED, DL-BASED, AND FL-BASED NETWORK TRAFFIC CLASSIFICATION

Research	Dataset	Input Data	Field	Challenges Addressed		
				Encrypted Traffic	Privacy Protection	Model Unlearning
DPI [70]	Self-collected dataset	Raw packets	Rule-based	✗	✗	✗
Second-order MC [33]	SSL/TLS sessions	Length + timing features	ML-based	✓	✗	✗
Constrained Clustering [35]	Real-world traffic traces	Flow statistics	ML-based	✓	✗	✗
Distiller [71]	ISCX VPN-nonVPN	L4 payload (784 B) 4 fields (32 packets)	DL-based	✓	✗	✗
FlowPic [44]	ISCX VPN-nonVPN	Flow images	DL-based	✓	✗	✗
Deep Packet [46]	ISCX VPN-nonVPN	Raw payloads	DL-based	✓	✗	✗
FS-Net [49]	Real-world traffic	Flow sequences	DL-based	✓	✗	✗
Tree-RNN [52]	ISCX dataset	Flow packets	DL-based	✓	✗	✗
Adversarial NTC [53]	Real-world traffic	Perturbed flows	DL-based	✓	✗	✗
ASGAFGNN [61]	CICIDS2017, NSL-KDD	Flow + temporal features	FL-based	✓	✓	✗
FEAT [62]	QUIC, ISCX	Flow features	FL-based	✓	✓	✗
FedeEDI [64]	IoT device traffic	Flow features	FL-based	✓	✓	✗
Fed-anids [67]	USTC-TFC2016, CIC-IDS2017, 2018	Flow features	FL-based	✓	✓	✗
DR-NTC [68]	ISCXVPN2016	Flow features	FL-based	✓	✓	✗
FedSL-LSTM [66]	STIN sequential data	Time-series signals	FL-based	✓	✓	✗
Amnesiac [28]	MNIST, CIFAR100	Images	FMU-based	✗	✓	✓
FedAU [72]	MNIST, CIFAR100, CIFAR10	Images	FMU-based	✗	✓	✓
FedRecovery [73]	MNIST, CIFAR10, SVHN, USPS	Images	FMU-based	✗	✓	✓
FedUnSilApp (Ours)	NJUPT, MIRAGE, CIC-IoT-2022	Flow features	FMU-based	✓	✓	✓

some performance degradation and depends on the quality of the original global model, and its adaptability to unknown attacks may require further investigation. This paper [81] proposes an innovative unlearning method that combines Momentum Decay (MoDe) and memory-guided strategies to efficiently unlearn specific data in FL environments. Through a two-step decomposition including knowledge erasure and memory guidance, this method not only addresses performance degradation in unlearning but also significantly enhances execution efficiency, enabling rapid response to data removal requests. Compared with traditional retraining methods, this approach greatly improves computational efficiency while preserving model performance, making it especially suitable for handling complex unlearning tasks. Additionally, this method can effectively counter data poisoning attacks, further enhancing the security of FL.

E. Comparative Summary

Table I systematically compares the evolutionary trajectory of NTC technologies, tracing their development from traditional rule-based and ML-based methods to DL-based, FL-based, and FMU-based paradigms.

The rapid growth of encrypted traffic has rendered traditional port-based identification and DPI increasingly ineffective. ML-based NTC methods introduce statistical learning to improve encrypted traffic recognition but still rely heavily on handcrafted features, limiting scalability and their ability to capture complex feature correlations. With the emergence of DL, end-to-end models achieve substantial performance gains by learning representations directly from raw traffic. However, their reliance on centralised training raises serious concerns regarding privacy and data sovereignty.

FL addresses these limitations by enabling privacy-preserving collaborative training without sharing raw data. Recent FL-based NTC approaches demonstrate strong scalability and robustness across heterogeneous and large-scale environments.

Building upon FL, FMU has recently emerged to support the ‘right to be forgotten’, allowing efficient removal of specific data or clients while preserving model utility. Motivated by these advances, FedUnSilApp integrates encrypted traffic classification, privacy preservation, and unlearning into a unified federated framework tailored to the challenges posed by silent applications.

III. PRELIMINARIES

A. Federated Learning

FL is a distributed machine learning approach that allows multiple clients (e.g., mobile devices, edge servers) to collaboratively train a global model while keeping their local data decentralized [82]. This method is especially useful for privacy-sensitive applications, as it enables clients to contribute to the model without sharing raw data. Instead, only model updates are communicated to a central server for aggregation [83].

In a typical FL setup, we assume there are K clients, each with its own local dataset $\mathcal{D}_k$, where $k \in \{1, 2, \dots, K\}$. The objective of FL is to minimize a global loss function $F(w)$ across all clients. This objective can be formulated as:

$$F(w) = \sum_{k=1}^K \frac{n_k}{n} f_k(w), \quad (1)$$

where w represents the global model parameters, $n_k = |\mathcal{D}_k|$ is the number of data samples held by *client* _{k} , $n = \sum_{k=1}^K n_k$ is the total number of data samples across all clients, $f_k(w)$ is the local objective function (i.e., loss function) of *client* _{k} , defined as:

$$f_k(w) = \frac{1}{n_k} \sum_{(x_i, y_i) \in \mathcal{D}_k} \ell(w; x_i, y_i), \quad (2)$$

where $\ell(w; x_i, y_i)$ is the loss for a data sample (x_i, y_i) in $client_k$'s dataset, based on the model parameters w .

One of the most common algorithms for FL is Federated Averaging (FedAvg), which operates in iterative communication rounds. In each round:

- 1) The server sends the current global model parameters w_t to a set of clients.
- 2) Each selected $client_k$ performs local training by minimizing $f_k(w)$ on its dataset $\mathcal{D}_k$ and computes updated model parameters $w_k^{(t+1)}$ after multiple local epochs.
- 3) The server aggregates the updated model parameters from all participating clients to obtain a new global model w_{t+1} . This aggregation can be expressed as:

$$w_{t+1} = \sum_{k=1}^K \frac{n_k}{n} w_k^{(t+1)}, \quad (3)$$

This weighted averaging ensures that clients with larger datasets contribute more to the global model update. The FedAvg algorithm enables the global model to gradually improve by learning from decentralized data while maintaining data privacy.

B. Machine Unlearning

Machine Unlearning refers to the process of selectively removing the influence of certain data points from a trained machine learning model [84], [85]. This technique is particularly useful in privacy-sensitive applications, where users may request their data to be forgotten, or when erroneous data needs to be removed to improve model accuracy. The goal of unlearning is to modify the model so that it no longer retains any knowledge derived from the specific data points in question, without requiring a full retraining from scratch.

Suppose we have a machine learning model trained on a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, where w represents the model parameters, x_i are the input features, and y_i are the target labels. The model is trained by minimizing a loss function $\mathcal{L}$ over the dataset:

$$\min_w \mathcal{L}(w, \mathcal{D}) = \frac{1}{N} \sum_{i=1}^N \ell(w; x_i, y_i), \quad (4)$$

where $\ell(w; x_i, y_i)$ is the loss for a single data point (x_i, y_i) .

In the context of machine unlearning, let $\mathcal{D}^u \subset \mathcal{D}$ represent the subset of data to be 'forgotten' by the model, and let $\mathcal{D}^r = \mathcal{D} \setminus \mathcal{D}^u$ be the remaining data. The objective of machine unlearning is to modify the model such that it closely approximates a model trained only on $\mathcal{D}^r$, i.e., the model that would have been obtained if $\mathcal{D}^u$ had never been included in the training data.

C. Problem Formulation

This paper focuses on addressing the impact of the silent application in the field of NTC. In order to provide a clear and comprehensible explanation, the following definitions are given:

The packet set is formed by extracting TCP and UDP packets from application traffic. The packets are grouped into flows using quintuple features, recorded in a hash table with flow durations capped at 120 seconds. Session characteristics, including the source-destination IPs, are used to define flow direction. Flow features are extracted bi-directionally from forward and backward flows Se , with FF representing flow statistical features.

TABLE II
LIST OF ABBREVIATIONS

Notation	Explanation
Se	A set of session flows
FF	The set of features of a bi-directional flow.
sFF	Scaled set of features of a bi-directional flow.
x	A batch of sFF serves as input to the model.
y	Label of the flow feature.
K	Total number of clients
k_i	The i -th client initiating the forgetting request.
$\mathcal{D}_i$	Dataset of the i -th client.
$\mathcal{D}_{k_i}^u$	Data of the client initiating forgetting that needs to be removed.
$\mathcal{D}_{k_i}^r$	The remaining data is needed in the client that initiated the forget request
S	Set of deleted label indices.
$\widehat{\mathcal{D}_{k_i}^r}$	The remaining data after label correction in the client that initiated the forget request.
PT_i	Feature extraction layer of the i -th client.
W_i^l	Classifier layer of the i -th regular client.
$W_{k_i}^\alpha$	Classifier layer of the k_i client requesting forgetting.
A	The original label set of the silent applications.
B	The new label set of the silent applications.
C	Set of class labels to be removed.
K_{forget}	Set of clients requesting forgetting.
$K_{\text{non-forget}}$	Set of other clients.
F_{avg}	Average parameters of PT .
$W_{\text{fc, non-forget}}$	Average classifier parameters of non-forgetting clients.
$W_{\text{fc, forget}}$	Average classifier parameters of forgetting clients.

These features are computed at the edge and divided into packet-level, flow-level, and statistical features. After feature scaling, FF is transformed into sFF , which serves as the model's fundamental input, represented as x .

A list of abbreviations is presented in Table II to facilitate understanding of the notations adopted in what follows.

IV. METHODOLOGY

A. Overview of FedUnSilApp

As illustrated in Fig. 3, FedUnSilApp introduces an FMU framework specifically designed to mitigate the impact of silent applications. Each client maintains a local model consisting of a feature extraction module PT and a classifier W , which are trained on locally collected and labeled traffic data. Under normal conditions, clients follow the standard FL workflow: local models are trained independently, and the corresponding parameters are periodically uploaded to the server for aggregation, resulting in a global feature extractor and classifier.

To support selective unlearning, FedUnSilApp augments the conventional learning pipeline with an auxiliary forgetting module W^α . When a client requests the removal of data associated with silent applications, a distillation-based forgetting process is triggered locally to suppress the classifier components correlated with silent applications, producing the forgetting module W^α . This mechanism enables targeted unlearning without retraining the model from scratch or disrupting representations learned from active applications.

At the server side, FedUnSilApp adopts a scalable and modular aggregation strategy. Feature extraction parameters PT from all clients are aggregated into a unified global representation, preserving shared traffic semantics. In contrast, classifier parameters are aggregated in a decoupled manner: parameters from regular clients (W^l) and those from clients performing

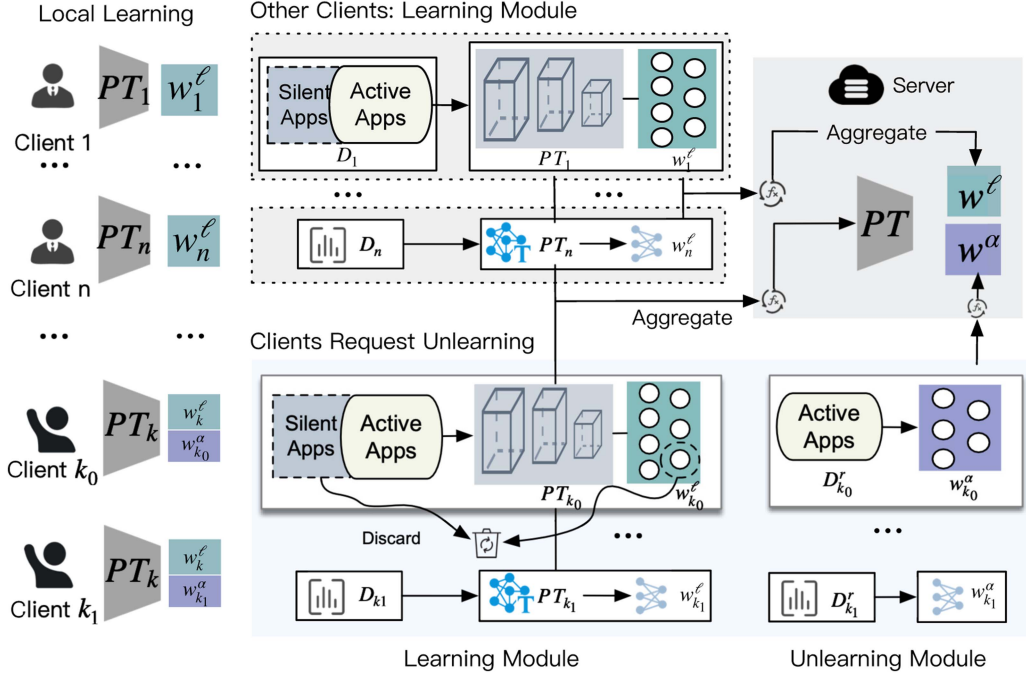


Fig. 3. Overview of the FedUnSilApp framework. During the learning stage, clients locally train models and share feature extractors for aggregation. When forgetting is requested, the unlearning module prunes silent-app neurons and retrain on remaining data before scalable aggregation merges both PT and W on the server.

unlearning (W^α) are treated separately. By jointly aggregating PT , W^l , and W^α , the server constructs global models that flexibly adapt to heterogeneous client states.

In our FedUnSilApp, a horizontal FL scenario is applied. We assume that a total of K clients conduct collaborative training to build a federated model $\omega = (PT, W)$ (PT is used to extract latent network traffic features, W is the classifier), which can be formulated as follows:

$$\min_{\omega} \sum_{k=1}^K \sum_{i=1}^{n_k} \frac{\ell(F_{PT,W}(x_{k,i}), y_{k,i})}{n_1 + \dots + n_K}, \quad (5)$$

where ℓ represents the loss function (e.g., cross-entropy loss). $D_k = \{(x_{k,i}, y_{k,i})\}_{i=1}^{n_k}$ is the dataset owned by $client_k$, with size n_k . When applying forgetting, client k_0 requests to remove the influence of its data from the federated model, i.e., a client-level forgetting request, aiming to exclude client k_0 's contribution from the learned parameters. If multiple clients submit forgetting requests simultaneously, we denote the set of requesting clients as K_{forget} .

B. Fedunsilapp

This paper focuses on how FMU addresses the issue of silent applications. Accordingly, we emphasise the workflow for a client requesting unlearning, as illustrated in Fig. 4. From the perspective of $client_k$, $client_k$ initially collects labeled traffic data in its local environment and extracts flow-level features to initialize and train its local model, as in conventional federated learning. When $client_k$ encounters a requirement to remove data associated with silent applications, it issues an unlearning request, upon which the local forgetting module is activated to selectively prune classifier neurons correlated with silent

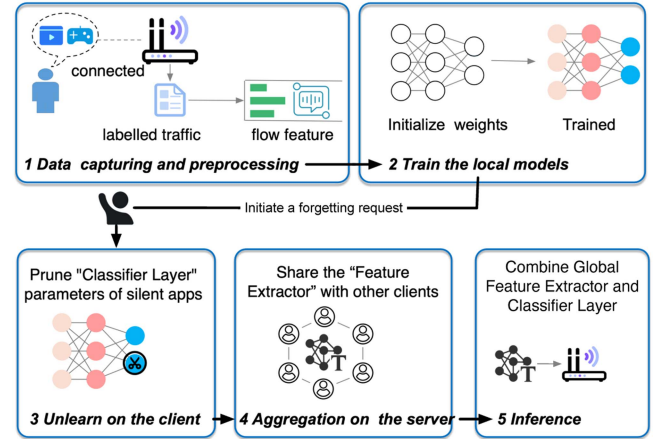


Fig. 4. The workflow of $client_k$. After model training on traffic data, a forgetting request triggers neuron pruning for silent applications on the client side, followed by feature extractor aggregation on the server to form the final global model for inference.

applications, thereby mitigating its negative impact on model stability. During global aggregation, all clients upload their feature extraction parameters PT , which are jointly aggregated to form a shared representation. Meanwhile, the classifier parameters of clients requesting unlearning (W^α) are decoupled from those of regular clients (W^l) and aggregated separately. This modular aggregation strategy enables FedUnSilApp to adapt to heterogeneous unlearning demands while preserving stability and scalability under dynamic application behaviors. In the following, we will respectively introduce the details of the modules and the training strategies.

follows:

$$F_{\text{avg}} = \frac{1}{K} \sum_{k=1}^K PT_k, \quad (9)$$

In this step, we iterate over each client $k = 1, \dots, K$, and perform a summation for each client's feature layer parameter PT_k , then divide the aggregated result by the total number of clients K to obtain the average value. This operation ensures that the aggregated feature parameter F_{avg} is the average of the feature parameters from all clients, regardless of whether they are forgetting clients or non-forgetting clients.

Aggregation of FC layers from non-forgetting clients: Only the FC parameters from the non-forgetting clients are aggregated to obtain the average FC parameter $W_{\text{fc, non-forget}}$. The specific formula is as follows:

$$W_{\text{fc, non-forget}} = \frac{1}{|K_{\text{non-forget}}|} \sum_{k \in K_{\text{non-forget}}} W_k, \quad (10)$$

In this step, we iterate over each client k from the non-forgetting client set $K_{\text{non-forget}}$, perform a summation for each FC parameter W_k , then divide the aggregated result by the total number of non-forgetting clients $|K_{\text{non-forget}}|$ to obtain the average value. This operation ensures that $W_{\text{fc, non-forget}}$ is the average of the FC parameters from all non-forgetting clients, and that the FC layers of forgetting clients are not involved in the aggregation.

Aggregation of FC layers from forgetting clients: Only the FC parameters from the forgetting clients are aggregated to obtain the average FC parameter $W_{\text{fc, forget}}$. The specific formula is as follows:

$$W_{\text{fc, forget}} = \frac{1}{|K_{\text{forget}}|} \sum_{k \in K_{\text{forget}}} W_k, \quad (11)$$

In this step, we iterate over each client k from the forgetting client set K_{forget} , perform a summation for each FC parameter W_k , then divide the aggregated result by the total number of forgetting clients $|K_{\text{forget}}|$ to obtain the average value. This ensures that $W_{\text{fc, forget}}$ is the average of the FC parameters from all forgetting clients, and the FC layers of non-forgetting clients are not involved in the aggregation.

Finally, we combine the previously calculated average feature parameter F_{avg} with different FC layer aggregation results to form two global models. Model $w_{\text{non-forget}}$ is composed of the feature parameter F_{avg} obtained from all clients and the FC layer aggregation result $W_{\text{fc, non-forget}}$ from non-forgetting clients, i.e.,

$$w_{\text{non-forget}} = (F_{\text{avg}}, W_{\text{fc, non-forget}}), \quad (12)$$

Model w_{forget} is composed of the feature parameter F_{avg} obtained from all clients and the FC layer aggregation result $W_{\text{fc, forget}}$ from forgetting clients, i.e.,

$$w_{\text{forget}} = (F_{\text{avg}}, W_{\text{fc, forget}}), \quad (13)$$

$w_{\text{non-forget}}$ and w_{forget} serve different purposes, depending on whether the information from forgetting clients is included. This scalable dual-path aggregation mechanism enables the global model to adapt to dynamic application states and supports model expansion when silent applications become active again.

4) *Training and Unlearning Procedure*: Algorithm 1 summarizes the overall workflow of FedUnSilApp, consisting of three modules: the learning module, the unlearning (forgetting) module, and the scalable aggregation module. At the beginning

Algorithm 1: Unlearning Silent Apps in FL(Learning Module, Unlearning Module and Scalable Aggregation Module).

```

1: Input: Communication rounds  $T$ , Client number  $K$ ,
   Dataset  $D_k$ , Forgetting clients  $K_{\text{forget}}$ ;
2: Initialize: Network traffic features extractor  $PT$ ,
   learning model  $W^l$ , forgetting model  $W^\alpha$ ;
3: for client  $k$  in  $\{1, \dots, K\}$  do
4:   Clients perform:
5:   for  $t = 1, 2, \dots, T$  do
6:     Set  $PT_k = PT$ ,  $W_k^l = W^l$ 
7:     Calculate the loss:  $\hat{\ell} = \ell(D_k; PT_k, W_k^l)$ 
8:      $W_k^l \leftarrow W_k^l - \eta \nabla W_k^l \hat{\ell}$ 
9:      $PT_k \leftarrow PT_k - \eta \nabla PT_k \hat{\ell}$ 
10:   end for
11:   if  $k \in K_{\text{forget}}$  then
12:     for  $t = 1, 2, \dots, T$  do
13:       Set  $D_k^r = D_k - D_k^u = \{x \mid x \in D_k \wedge x \notin D_k^u\}$ 
14:       Set  $\hat{D}_k^r = g(D_k^r)$ 
15:       Set  $W_k^\alpha = W^\alpha$ 
16:       Freeze  $PT_k$ 
17:       Calculate the loss:  $\hat{\ell} = \ell(\hat{D}_k^r; W_k^\alpha)$ 
18:        $W_k^\alpha \leftarrow W_k^\alpha - \eta \nabla W_k^\alpha \hat{\ell}$ 
19:     end for
20:   end if
21: end for
22: Upload  $W_k^\alpha$ ,  $W_k^l$ , and  $PT_k$  to the server
23: Server aggregates:
24:  $PT = \frac{1}{K} \sum_{k=1}^K PT_k$ 
25:  $W_{\text{fc, non-forget}} = \frac{1}{|K_{\text{non-forget}}|} \sum_{k \in K_{\text{non-forget}}} W_k^l$ 
26:  $W_{\text{fc, forget}} = \frac{1}{|K_{\text{forget}}|} \sum_{k \in K_{\text{forget}}} W_k^\alpha$ 
27: return  $PT$ ,  $W_{\text{fc, non-forget}}$ ,  $W_{\text{fc, forget}}$ 

```

of each communication round, all clients participate in the learning process. Each client locally trains its model using private traffic data to update both the feature extractor and classifier parameters. After local optimization, the updated local models PT_k and W_k^l are uploaded to the server.

When silent applications are detected, only clients requiring forgetting enter the unlearning module. These clients remove the data samples corresponding to silent applications and perform neuron pruning on the classifier while freezing the feature extractor to preserve previously learned general representations. The forgetting model W_k^α is then fine-tuned on the remaining data, ensuring that the contribution of silent applications is erased from the decision layer while minimizing performance degradation on active applications.

Next, the scalable aggregation module updates the global model by aggregating uploaded parameters. Unlike classical FL aggregation, FedUnSilApp aggregates the feature extraction layers from all clients to maintain global generalization, while the discriminative layers are aggregated separately for forgetting clients and non-forgetting clients. Finally, the server distributes the updated global feature extractor and discriminative heads to all clients, forming the next round of communication. This process continues until convergence.

5) *Theoretical Analysis*: In the entire forgetting process, we need to meet two requirements: The first requirement is that the forgetting operation should not affect the model's accuracy on

the retained data. Specifically, the logit vector of model W^α on the retained data should be consistent with the original model W , i.e.,

$$\arg\max_i F_{PT,W^l}^i(x) = \arg\max_{g^{-1}(i)} F_{PT,W^\alpha}^{g^{-1}(i)}(\hat{x}),$$

$$x \in D^r, \quad \hat{x} \in \widehat{D}^r, \quad (14)$$

where $F_{PT,W}(x)$ represents the logit vector of the well-trained model W for input x , i.e., $[o_0, o_1, \dots, o_n]$. $F_{PT,W}^i(x)$ represents the i -th logit value. $F_{PT,W^\alpha}(x)$ represents the logit vector of the well-trained model W^α for input $\hat{x}$, i.e., $[o_0, o_1, \dots, o_{n-j}]$. $F_{PT,W^\alpha}^i(x)$ represents the i -th logit value. Here, j represents the number of categories of silent applications. As shown in Fig. 5, we calculate the loss between the active applications in the previous model and the new model. The loss is calculated as follows:

$$p_k = \frac{e^{o_k}}{\sum_{t=0}^{n-j} e^{o_t}}, \quad (15)$$

where o_k denotes the k -th logit in $F_{PT,W^\alpha}(\hat{x})$ (restricted to the remaining $n-j$ classes).

$$\log p_k = \log \left(\frac{e^{o_k}}{\sum_{t=0}^{n-j} e^{o_t}} \right), \quad (16)$$

$$\ell = -\frac{1}{BS} \sum_{i=1}^{BS} \sum_{k=0}^{n-j} y_{ik} \log p_{ik}, \quad (17)$$

Here, p_{ik} is the predicted probability of class k for sample i , and y_{ik} denotes the (possibly soft) target label distribution. BS is the number of samples trained in a batch.

The second requirement is that the model after forgetting should exhibit incorrect classification behavior on the forgotten data D^u , as described by [Chen et al. 87]. Specifically, the logit output of model W^α should differ from the original label of the forgotten data D^u , i.e.,

$$\arg\max F_{PT,W^\alpha}^{g^{-1}(i)}(x) \neq y, \quad x \in D^u. \quad (18)$$

where y is the true label. In other words, this requirement ensures that the model after forgetting should not retain information about the forgotten data D^u . In the next section, we will validate our approach through experiments.

V. EVALUATIONS

A. Experiment Goal

Fig. 6 illustrates the comprehensive experimental workflow designed to validate the effectiveness of FedUnSilApp. This workflow comprises a data preparation phase and three primary experimental stages. Evaluations were conducted across four datasets: NJUPT, MIRAGE, CIC-IoT 2022 and USTC-TFC 2016. Each dataset was preprocessed and partitioned into training and testing sets. All experiments were conducted from the perspective of a client initiating learn-to-request operations. To simulate real-world deployment scenarios, we assumed that two specific applications were missing from each dataset. Consequently, the test sets were deliberately designed to exclude these applications, ensuring that the model evaluations would reflect situations in which certain applications were unavailable or inactive. The experimental evaluation comprises three experiments

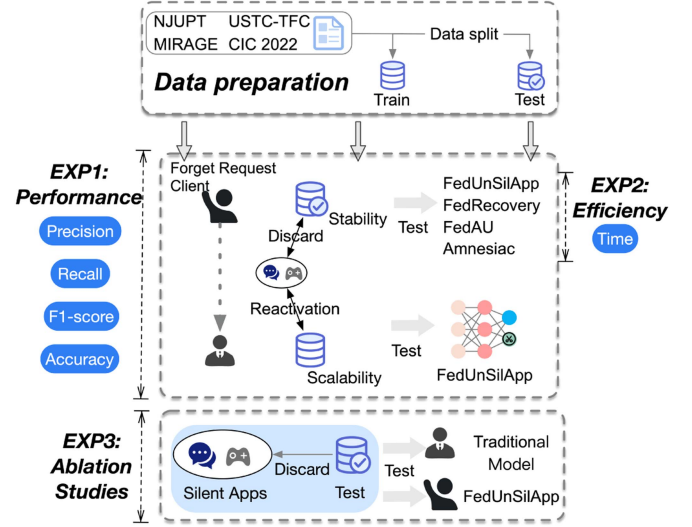


Fig. 6. The design process of the experiment.

labelled EXP1, EXP2, and EXP3, each of which aims at a distinct aspect of the proposed method. Specifically, EXP1 focuses on classification accuracy and robustness; EXP2 evaluates the efficiency and training time; and EXP3 investigates the contribution of individual components through ablation studies.

- 1) **EXP1: Performance Comparison.** Our focus is on comparing the performance of FedUnSilApp with that of representative State-Of-The-Art (SOTA) FMU methods, including FedRecovery, FedAU and Amnesiac. The objective is to demonstrate the effectiveness of FedUnSilApp in classification tasks under unlearning scenarios. In addition to evaluating overall accuracy, we assess the stability of the proposed method across different application categories. To examine scalability further, we evaluate the capability of FedUnSilApp to rapidly restore accurate recognition after unlearning by simulating scenarios in which previously silent applications are reactivated. This analysis enables us to verify whether the model can efficiently adapt to dynamic transitions in the application state without compromising classification performance.
- 2) **EXP2: Efficiency and Training Time Analysis.** EXP2 evaluates the efficiency of different State-Of-The-Art (SOTA) methods by measuring their unlearning-related training time. For FedUnSilApp, we reported the per-client training time and unlearning time separately, as practical federated deployments require unlearning updates to be propagated and applied to clients promptly. The aim of this experiment is to assess whether FedUnSilApp can achieve rapid adaptation while maintaining high performance, making it suitable for large-scale real-world deployments.
- 3) **EXP3: Ablation Studies.** We conducted an ablation study to isolate and quantify the contribution of the unlearning mechanism within FedUnSilApp. In the first stage, multiple clients trained local models independently using their respective datasets. These locally trained models were then combined to create a global model. We then evaluated the performance of this global model and each client model using a predefined test set to determine the baseline behaviour. The aim was to verify that the global

TABLE III
THE NUMBER OF CATEGORIES IN THE DATASET IS PRESENTED

MIRAGE		NJUPT	
applications	amount	applications	amount
Accuweather	3991	QQ music	39465
Contextlogic	2792	Garena	21872
Dropbox	2635	VR	33611
Duolingo	3922	background	62228
Facebook	3195	MOOC	14842
Groupon	839	Peacekeeper	12120
Hypah	1583	bilibili	20014
Iconology	2806	Weibo	22061
Joelapenna	3443	TikTok	16640
Motain	5215	DOYU	15821
Pinterest	2152	NARUTO	27240
Spotify	3514	iQIYI	36740
Subito	3538	HOK	42734
Trello	1310	Tieba	18206
Tripadvisor	1260	ZH	16643
Twitter	2407	TencentVideo	11913
Viber	1112	LoLWR	19841
Waze	5278	HUYA	108757
Youtube	2507	TFT	23718

model's classification was accurate. In the second stage, we simulated a realistic 'forgetting' scenario in which *client*₂ requested the deletion of data relating to the silent application. By comparing the model with and without the forgetting module, we tested the model's accuracy directly and used the confusion matrix to observe changes in its performance specifically. Thus, we verified the effectiveness of the FedUnSilApp framework.

Finally, all experimental outcomes are evaluated using a unified set of performance metrics, including training time, accuracy, recall, F1-score, and precision. This multifaceted evaluation framework ensures a thorough assessment of FedUnSilApp's performance and advantages across various dimensions.

B. Datasets

In this experiment, we selected four datasets: MIRAGE, NJUPT, CIC IoT Dataset 2022, and USTC-TFC 2016. The MIRAGE dataset [88] is specifically designed for mobile traffic analysis and contains real-world data associated with mobile applications. It encompasses multiple domains, including social media, video, music, gaming, and news. Each traffic packet within the dataset is annotated with comprehensive information such as timestamps, source and destination addresses, protocols, and packet lengths. Complementing this public dataset, the private NJUPT dataset used in this study was collected within a campus network environment and comprises traffic from 19 popular applications. As detailed in Table III, these applications cover diverse categories, including video streaming, music, gaming, and social media platforms. The NJUPT dataset includes 91 distinct features per sample. To evaluate the generalization capability of our approach across heterogeneous network environments, we additionally incorporated the CIC IoT Dataset 2022 [89] and USTC-TFC 2016 [90]. For these two datasets, we selected the top ten categories with the largest number of instances to train and validate our method. All traffic samples across the four datasets are uniformly labeled according to application type, providing clear ground truth for supervised learning. By integrating these diverse datasets into our experiments, we aim to validate the effectiveness and robustness

of our FMU framework under heterogeneous encrypted traffic scenarios.

- **MIRAGE:** This dataset primarily features applications from categories such as social networking, travel, weather, and basic utilities, including prominent platforms like Facebook, Twitter, Pinterest, AccuWeather, and Tripadvisor. The prevalence of these frequently used daily-life applications indicates that MIRAGE focuses on capturing typical user behavior patterns during routine activities. The sample size per application in MIRAGE is relatively uniform, mostly ranging between 1,000 and 5,000. In our experiments, we employed the SMOTE data augmentation technique to expand each major application class to 11,000 samples for subsequent training and testing phases.
- **NJUPT:** This dataset contains a higher proportion of modern entertainment and media consumption platforms, such as QQ Music, Bilibili, Weibo, TikTok, iQIYI, Huya, and Tencent Video, with a predominant focus on video, music, and gaming applications. Notably, NJUPT includes a substantial volume of traffic samples related to background processes (e.g., the Background category contains 62,228 samples), highlighting the dataset's emphasis on capturing patterns of background operations alongside active user interactions. The sample size distribution in NJUPT exhibits marked disparities, with certain categories (e.g., Background, VR, and TFT) significantly outnumbering others. This characteristic makes NJUPT particularly suitable for analyzing application usage intensity and user preferences under imbalanced data conditions.
- **CIC IoT Dataset 2022:** This publicly available dataset, developed and released by the Canadian Institute for Cybersecurity, is specifically tailored for IoT security research. Collected within a controlled laboratory environment, it contains authentic network traffic generated by various modern smart home devices, including smart lights, smart plugs, surveillance cameras, doorbells, and voice assistants. The dataset provides rich network traffic metadata alongside raw packet capture files, with all data meticulously labeled to indicate whether the traffic is benign or belongs to a specific attack category.
- **USTC-TFC 2016:** This dataset, curated by the University of Science and Technology of China (USTC), is widely used in encrypted traffic classification and intrusion detection research. It contains real-world traffic traces collected from a diverse range of mainstream applications, covering categories such as chat, email, streaming media, and file transfer.

C. Experimental Settings

Our experimental code runs on our server. In this paper, Python3 is the main programming language. The baseline environment specifications of our server include CPU, Graphics Card, Python, CUDA, System, and PyTorch information; Specific parameters are shown in Table IV.

Since this experiment focuses on the impact of accuracy and consumption from the perspective of silent applications, we do not consider other hardware indicators. We evaluated the selected classifiers using four key performance metrics—Precision, Accuracy, Recall, and F1 Score—to offer a thorough perspective on their effectiveness in classification tasks.

TABLE IV
EXPERIMENTAL EQUIPMENT

Name	Specification
CPU	Intel(R) Xeon(R) Platinum 8378C
Cuda Version	12.6
Graphics Card	NVIDIA GeForce RTX 4090
System Version	20.04.1-Ubuntu
Pytorch Version	2.6.0
Python Version	3.10

D. Baseline Methods

Amnesiac [28] addresses the machine unlearning problem under the GDPR ‘Right to be Forgotten’ requirement. It focuses on removing the influence of specific training data from deep neural networks to mitigate model inversion and membership inference attacks. By introducing unlearning and amnesiac unlearning strategy, it enables model owners to erase sensitive information while preserving model utility, ensuring compliance with data deletion requests.

FedRecovery [73] is designed to achieve efficient FMU without relying on retraining. It removes a client’s influence by subtracting weighted gradient residuals from the global model, and incorporates differential privacy with Gaussian noise to ensure that the unlearned model remains difficult to distinguish from a retrained one. The method operates without convexity assumptions and effectively supports neural network training scenarios.

FedAU [72] proposes a lightweight federated unlearning framework that introduces an auxiliary unlearning module into the training pipeline. It performs data removal through a simple linear operation, avoiding expensive post-processing steps and supporting multi-client and multi-granularity unlearning, including sample-level, class-level, and client-level removal. Experiments on MNIST, CIFAR-10 and CIFAR-100 show effective unlearning with minimal accuracy degradation.

E. EXP1: Performance Comparison

1) *Analysis of Scalability and Stability*: To assess the stability of FedUnSilApp, we analyze the per-class classification performance of active applications on the NJUPT and MIRAGE datasets after applying the proposed unlearning procedure to remove the influence of silent applications. As shown in Table VI, the overall accuracy on active classes reaches 0.8878, indicating that the proposed model maintains relatively consistent performance in different applications without significant class-level performance degradation. Notably, Bilibili and Peacekeeper achieve particularly strong results in terms of precision and recall, demonstrating a robust discriminative capability. In contrast, the model exhibits comparatively weaker performance on the Background and TikTok categories, with a lower recall observed for TikTok. This can be attributed to the fact that TikTok traffic contains a large volume of short-form video media flows whose traffic patterns are highly dynamic and less distinctive, increasing the likelihood of missed detections. For the MIRAGE dataset, Table V shows that the Joelapenna category achieves consistently high precision and recall, indicating accurate and comprehensive recognition of samples within this class. Similarly, the Motain category demonstrates near-ideal classification performance. For the Iconology and Pinterest categories, although both precision and recall remain

TABLE V
CLASSIFICATION RESULTS ON THE MIRAGE DATASET

Class	Precision	Recall	F1-Score	Support
Facebook	0.8661	0.7616	0.8105	994
Groupon	0.7894	0.8729	0.8291	992
Hypah	0.8739	0.8880	0.8809	991
Iconology	0.7786	0.9353	0.8498	989
Joelapenna	0.8781	0.8975	0.8877	995
Motain	0.9017	0.9008	0.9013	988
Pinterest	0.9037	0.7821	0.8385	996
Spotify	0.8930	0.8143	0.8517	991
Accuracy	0.8565			

TABLE VI
CLASSIFICATION RESULTS ON THE NJUPT DATASET

Class	Precision	Recall	F1-Score	Support
VR	0.8184	0.9133	0.8633	992
background	0.8439	0.6893	0.7588	988
MOOC	0.8475	0.9425	0.8925	991
Peacekeeper	0.9167	0.9337	0.9248	995
bilibili	0.9770	0.9829	0.9799	992
Weibo	0.9171	0.9577	0.9369	993
TikTok	0.8375	0.7067	0.7665	992
DOYU	0.8888	0.9255	0.9068	993
Accuracy	0.8878			

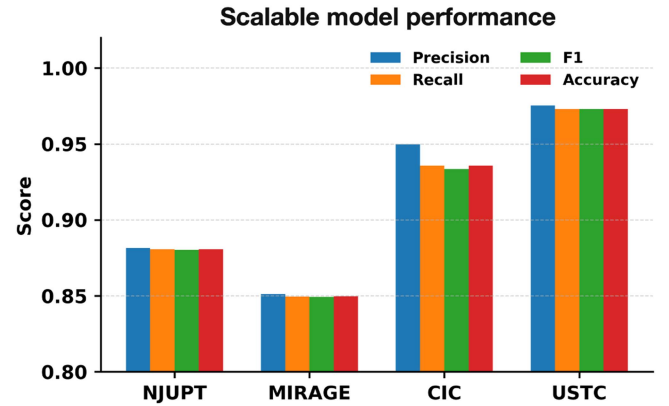


Fig. 7. Scalability performance of the proposed model across four datasets.

at satisfactory levels, a small number of misclassifications and missed detections are still observed, suggesting residual inter-class similarity in traffic patterns. To validate the scalability of our model, we further performed new experiments by assuming that the silent applications in the original datasets become active. Based on this scenario, we evaluate the performance of FedUnSilApp and observe its scalability under the silent-to-active transition. Fig. 7 presents the scalability evaluation when silent applications transition to active states. The model retains stable Precision, Recall, F1 and Accuracy across all datasets, demonstrating that it can scale effectively without significant performance degradation under application-state changes. As shown in Fig. 8, the model maintains consistent Precision, Recall and F1 across different traffic sources, including QQ Music and Garena in NJUPT, Dropbox and Duolingo in MIRAGE, Camera amcrest and Camera arloqcam in CIC-IoT 2022, and BitTorrent and Gmail in USTC-TFC 2016.

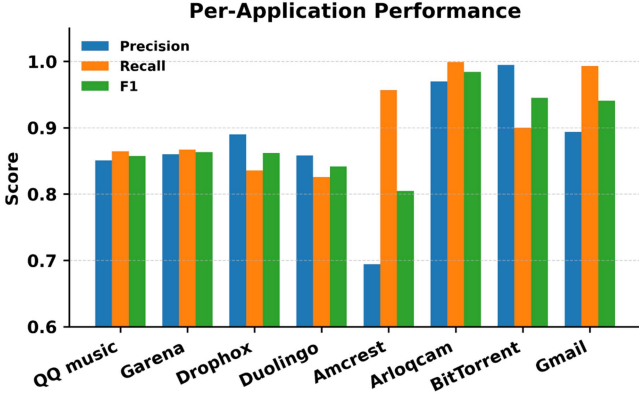


Fig. 8. Scalability performance of the proposed model across four datasets.

TABLE VII

SCALABILITY EVALUATION OF THE PROPOSED MODEL DURING THE TRANSITION FROM SILENT TO ACTIVE APPLICATIONS ACROSS MULTIPLE DATASETS

Dataset	Precision	Recall	F1	Accuracy
NJUPT	0.8816	0.8806	0.8802	0.8806
MIRAGE	0.8511	0.8495	0.8493	0.8496
CIC-IoT 2022	0.9497	0.9357	0.9334	0.9357
USTC-TFC 2016	0.9753	0.9731	0.9731	0.9731

TABLE VIII

FINE-GRAINED SCALABILITY EVALUATION OF THE PROPOSED MODEL UNDER THE SILENT-TO-ACTIVE TRANSITION SCENARIO

Dataset	Silent→Active Apps	Precision	Recall	F1
NJUPT	QQ music	0.8506	0.8643	0.8574
	Garena	0.8600	0.8669	0.8634
MIRAGE	Dropbox	0.8901	0.8357	0.8620
	Duolingo	0.8583	0.8257	0.8417
CIC-IoT 2022	Camera_amcrest	0.6940	0.9569	0.8045
	Camera_arloqcam	0.9699	0.9990	0.9842
USTC-TFC 2016	BitTorrent	0.9945	0.8998	0.9448
	Gmail	0.8935	0.9930	0.9406

Table VII evaluates the overall performance of FedUnSilApp in detecting scenarios where silent applications are rapidly re-activated as active applications across four datasets. Specifically, the reported results correspond to the aggregated performance over ten applications. As shown in Table VII, the proposed model maintains stable performance under application state transitions, achieving F1 scores of 0.8802 and 0.8493 on NJUPT and MIRAGE, respectively, while reaching 0.9334 and 0.9731 on CIC-IoT 2022 and USTC-TFC 2016. Notably, the consistently high performance on CIC-IoT 2022 and USTC-TFC 2016 (with $F1 \geq 0.93$ and $Accuracy \geq 0.93$) indicates that FedUnSilApp exhibits strong scalability and robustness when applications undergo rapid state transitions from silent to active. This suggests that the proposed unlearning mechanism does not compromise the model's ability to adapt to evolving application behaviors.

Table VIII further provides a fine-grained scalability analysis at the application level by examining the re-activation accuracy of individual applications. For each dataset, two silent applications are re-activated to simulate local state transition scenarios. On the MIRAGE, NJUPT, and USTC-TFC 2016 datasets, both re-activated applications achieve stable and consistent accuracy, demonstrating reliable recovery performance. On the CIC-IoT

TABLE IX
PERFORMANCE COMPARISON OF DIFFERENT FMU METHODS ACROSS DATASETS

Dataset	Method	Precision	Recall	F1	Accuracy
NJUPT	Amnesiac	0.7075	0.5933	0.6334	0.7417
	FedAU	0.8252	0.8171	0.8153	0.8170
	FedRecovery	0.6566	0.6060	0.6270	0.7577
	FedUnSilApp	0.8808	0.8814	0.8787	0.8816
MIRAGE	Amnesiac	0.6672	0.3932	0.4875	0.4917
	FedAU	0.7066	0.7007	0.7010	0.7007
	FedRecovery	0.5093	0.4579	0.4765	0.5725
	FedUnSilApp	0.8639	0.8625	0.8625	0.8625
CIC-IoT 2022	Amnesiac	0.6607	0.4827	0.4745	0.6036
	FedAU	0.9301	0.8764	0.8406	0.8763
	FedRecovery	0.7340	0.5029	0.4728	0.6288
	FedUnSilApp	0.9328	0.9139	0.9126	0.9138
USTC-TFC 2016	Amnesiac	0.7764	0.6954	0.7177	0.8944
	FedAU	0.9959	0.9958	0.9958	0.9958
	FedRecovery	0.7858	0.5676	0.6047	0.6493
	FedUnSilApp	0.9957	0.9957	0.9957	0.9957

2022 dataset, for the previously deactivated Camera arloqcam application, the model achieves a precision of 0.9699, a recall of 0.9990, and an F1-score of 0.9842 after re-activation, indicating that FedUnSilApp can rapidly restore high-quality recognition when such applications return to the system. In contrast, for the Camera amcrest application, the recall remains relatively high (0.9569), but the precision is lower (0.6940), leading to a reduced F1-score. This is likely because Amcrest camera traffic exhibits strong intra-class diversity, where background operations (e.g., keep-alive, cloud synchronization, and event-triggered transmissions) coexist with media streaming flows. The resulting variability weakens the discriminability of the class and encourages the classifier to produce more positive predictions for Amcrest-like patterns, which manifests as a higher false-positive rate and thus lower precision. Overall, these results indicate that FedUnSilApp can efficiently adapt to localized activation without noticeable performance degradation, demonstrating strong scalability when applications shift from silent to active status.

2) *Overall Accuracy Comparison With SOTA Methods:* As shown in Table IX, different methods exhibit substantial differences in classification accuracy across datasets. In the field of FMU, a common paradigm is to restore model utility by retraining or fine-tuning the model using the remaining data. FedRecovery follows this retraining-based approach and directly fine-tunes the model on retained data. While this strategy enables fast adaptation, it results in relatively limited classification performance. Specifically, FedRecovery achieves only about 76% accuracy on NJUPT, approximately 63% on CIC-IoT 2022, and 57% on MIRAGE, which is significantly lower than FedAU and FedUnSilApp. Amnesiac, which adopts gradient ascent for unlearning, shows even weaker performance on certain datasets. On MIRAGE, its accuracy drops below 50%, the lowest among all evaluated methods, indicating that aggressive gradient removal leads to severe degradation of discriminative capability. On USTC-TFC 2016, Amnesiac achieves moderate performance, but still falls short of the utility level required for high-fidelity encrypted traffic classification.

FedAU improves upon Amnesiac by enabling broader unlearning capabilities and achieving higher accuracy across datasets. It attains approximately 82% accuracy on NJUPT, around 70% on MIRAGE, and nearly 88% on CIC-IoT 2022,

TABLE X
TRAINING TIME COMPARISON OF DIFFERENT FMU METHODS ACROSS DATASETS

Method	NJUPT	MIRAGE	CIC-IoT 2022	USTC-TFC 2016
Amnesiac	167.71 s	213.24 s	165.83 s	138.44 s
FedAU	273.02 s	275.07 s	269.61 s	245.05 s
FedRecovery	33.21 s	34.11 s	33.18 s	21.66 s
FedUnSilApp	116.10 s	67.92 s	115.79 s	44.10 s

ranking second only to FedUnSilApp on CIC-IoT. However, FedAU does not explicitly address the misclassification of retained data as silent applications, which limits its robustness under application state transitions.

In contrast, FedUnSilApp consistently delivers the best overall performance across datasets. On the NJUPT dataset, it achieves the highest accuracy of approximately 88%, accompanied by strong precision and recall values of 0.8605 and 0.8566, respectively. On MIRAGE, FedUnSilApp reaches an accuracy of around 86%, significantly outperforming all competing methods. On CIC-IoT 2022, it attains the highest accuracy of 91%, demonstrating superior robustness under diverse application behaviors. On USTC-TFC 2016, FedUnSilApp and FedAU both achieve near-perfect performance, whereas FedRecovery and Amnesiac exhibit noticeable degradation, highlighting the effectiveness of FedUnSilApp in preserving model utility.

F. EXP2: Efficiency and Training Time Analysis

As shown in Table X, the evaluated methods exhibit clear trade-offs between training cost and model utility. FedRecovery achieves the shortest training time across all datasets, requiring only 33–34 seconds, as it directly fine-tunes the model using the remaining data without additional unlearning constraints. However, this efficiency comes at the cost of substantially reduced accuracy.

Amnesiac incurs significantly longer training times due to its gradient ascent-based unlearning process, which first removes learned representations and then aggregates updated models. This design introduces additional overhead, particularly for clients that did not initiate unlearning requests. As a result, Amnesiac’s training time approaches 3.5 minutes, while its accuracy remains relatively low. FedAU reduces unlearning time compared to Amnesiac but still requires a complex training process, including an initialization phase, multiple training rounds, and collaborative training of an auxiliary unlearning module by unlearning clients. These factors collectively lead to the longest training time among all methods, reaching nearly 4.5 minutes on NJUPT and approximately 275 seconds on MIRAGE. FedUnSilApp achieves a favorable balance between efficiency and performance. On NJUPT, it completes training in about 116 seconds while delivering the highest accuracy. On MIRAGE, its training time is approximately 68 seconds, which is nearly four times shorter than that of Amnesiac and FedAU. On CIC-IoT 2022, FedUnSilApp again significantly reduces training time (116 seconds) compared to FedAU, while achieving higher accuracy.

This efficiency advantage stems from FedUnSilApp’s scalable aggregation design, which separates the model into a *PT* and a lightweight classifier head (*FC*). The *PT*, containing the majority of model parameters, is trained and aggregated only

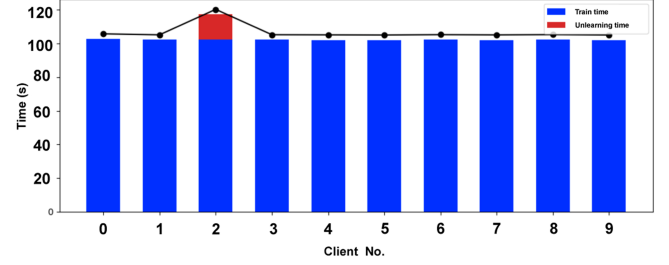


Fig. 9. The training time of each client. (Client No.2 is the client that initiated the forgetting request, and the red part in the bar chart represents the model’s forgetting time).

during the initial federated learning stage. When unlearning is triggered, the *PT* is frozen, and only the *FC* is updated by pruning and fine-tuning neurons associated with silent applications. Consequently, regular FL clients continue participating in global aggregation without interruption, while unlearning clients perform localized updates independently. This design significantly reduces retraining overhead and improves overall system efficiency and parallelism.

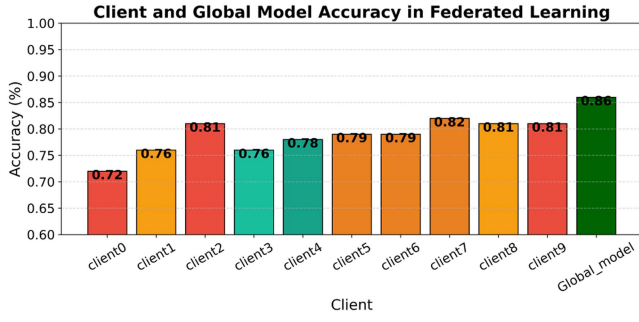
Although FedAU slightly outperforms other methods in terms of raw accuracy on USTC-TFC 2016, FedUnSilApp achieves nearly identical detection performance while dramatically reducing training cost (44.10 s vs. 245.05 s).

In addition, we measured the time (in seconds) required for each client (numbered 0–9) on the NJUPT dataset to complete model training and, where applicable, the unlearning operation, as shown in Fig. 9. From the “Train Time (s)” row, we observe that all clients require between 102.10 and 102.80 seconds for initial training—an average of approximately 102.26 seconds with a standard deviation of only 0.22 seconds—indicating that the workload is evenly distributed. It is worth noting that FedUnSilApp achieves efficient unlearning due to its decoupled architecture. The *PT* contains the majority of the parameters and is trained only once during the initial learning stage. When a client initiates an unlearning request, FedUnSilApp freezes the *PT* and updates only the lightweight classifier layer by pruning neurons associated with silent applications. As a result, only a small portion of the model is retrained. For example, *client*₂ completed unlearning in 13.76 seconds—merely 11.85% of its full training time—demonstrating that our approach maintains accuracy while significantly reducing computation overhead during unlearning.

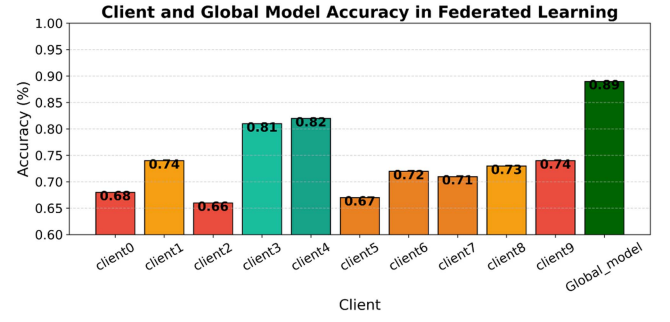
G. EXP3: Ablation Studies

As described in Experiment Goal, we assume that two classes of data have been removed from the dataset, and we evaluate our model on a test set that excludes those classes. This allows us to analyze the impact on model performance when the ‘forgetting module’ is enabled versus when it is not.

Firstly, we first verify the effectiveness of layer *PT* after aggregation, which can make the classifier more accurate. As shown in Fig. 10(a), the classification accuracy of each client on the MIRAGE dataset varies significantly. The client with the lowest accuracy is *client*₀, with an accuracy of 0.72, while *client*₂ and *client*₇ achieve the highest client accuracies, at 0.81 and 0.82, respectively. Overall, most clients have an accuracy ranging between 0.76 and 0.82, showing a certain degree of



(a) Client and global model accuracy on the MIRAGE dataset.



(b) Client and global model accuracy on the NJUPT dataset.

Fig. 10. Client and global model accuracy during federated learning on different datasets.

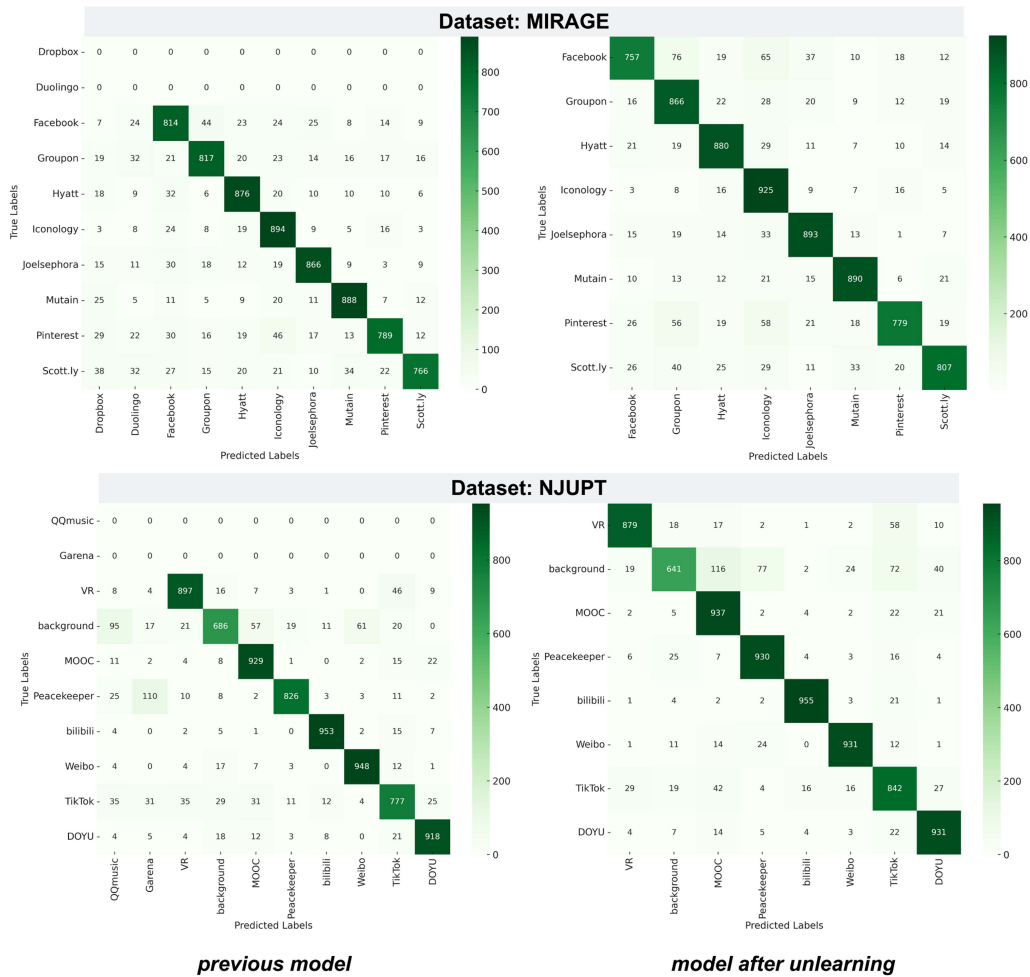


Fig. 11. Comparison of confusion matrices before and after applying the unlearning strategy on the MIRAGE and NJUPT datasets. The left two confusion matrices correspond to the conventional model evaluated on the two datasets without unlearning, whereas the right two matrices report the results obtained after applying the proposed unlearning strategy.

stability. Furthermore, the global model obtained through FL aggregation achieves the highest accuracy at 0.86, which is notably higher than the accuracy of any individual client. The results of this experiment show that the global model outperforms all client models with an accuracy of 0.89, indicating that the aggregate knowledge of each client through FL can improve the overall performance. As shown in Fig. 10(b), the accuracy

of the client model ranged from 0.66 (*client*₂) to 0.82 (*client*₄), reflecting significant differences in data distribution and training. Some clients, such as *client*₃ and *client*₄, perform close to the global model, suggesting high data quality or well-trained, while the lower accuracy of *client*₂ and *client*₅ (0.66 and 0.67, respectively) indicates possible bias, noise, or poorly trained data. Overall, the high performance of the global model validates

the effectiveness of the FL framework, which can improve prediction accuracy by aggregating multiple sources of data without accessing local data.

Furthermore, we verify the effectiveness of the weight matrix W^α after performing neurons unlearning operation. As shown in Fig. 11, confusion matrices on the left show the classification results of the original model, which has no unlearning mechanism. Confusion matrices on the right show the results after a subset of neurons associated with silent applications was deleted, which improved the overall classification accuracy. On the NJUPT dataset, the original model misclassifies a substantial amount of active application traffic as silent application classes. Specifically, it misclassifies 95 background-flow samples, 25 Peacekeeper samples and 35 TikTok samples as QQmusic samples. Additionally, 110 Peacekeeper samples and 31 TikTok samples are misclassified as Garena. These errors suggest that the model learns redundant or entangled features shared between background traffic and certain active applications. Introducing neuron-level unlearning weakens such redundant feature representations, effectively suppressing these erroneous predictions. Consequently, classification accuracy increases from 87.37% to 88.78%, corresponding to an absolute improvement of approximately 1.4 percentage points. A similar phenomenon is observed on the MIRAGE dataset when the unlearning strategy is disabled: a large number of active application samples are misclassified as silent applications. Notably, 38 Scott.ly samples and 29 Pinterest samples were incorrectly predicted as Dropbox, and 32 Groupon samples and 32 Scott.ly samples were misclassified as Duolingo. After enabling the neural unlearning mechanism, however, these misclassifications are substantially reduced. Consequently, classification accuracy improves from 84.56% to 85.66%, an absolute gain of approximately 1.10 percentage points.

Fig. 11 clearly demonstrates that without activating the client-side unlearning module, active application traffic can be erroneously misclassified as silent or background classes. Such misclassification can fragment the decision boundaries learned by the classifier, ultimately degrading downstream network functions, including traffic management and resource allocation. In contrast, the proposed neuron-level unlearning strategy effectively mitigates this issue by disentangling silent application features from active traffic representations.

VI. CONCLUSION

In this paper, we presented FedUnSilApp, a novel framework for FMU specifically designed to address the challenges posed by silent applications in NTC. Our approach combines the learning module, the unlearning module, and a scalable aggregation mechanism to ensure the stability and scalability of the model in dynamic data environments. FedUnSilApp effectively eliminates redundant or outdated information in the model through a selective neuron pruning mechanism guided by specific category features, while maintaining its performance. Such NTC require the ability to forget data on demand without compromising model stability. Through scalable aggregation, the framework enables quick recovery when previously silent applications become active again and need to be recognized.

Although FedUnSilApp addresses several key challenges in FL, there are still areas for future exploration. As FL increasingly integrates complex models, optimizing the scalability of FedUnSilApp and its compatibility with these architectures remains

a significant research direction. Finally, evaluating the framework's performance in real-world FL scenarios, where clients are heterogeneous and network conditions vary, is crucial for validating its robustness and efficiency in practical applications.

REFERENCES

- [1] G. Bovenzi, G. Aceto, D. Ciunzio, V. Persico, and A. Pescapé, "A Big Data-enabled hierarchical framework for traffic classification," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 4, pp. 2608–2619, Oct.–Dec. 2020.
- [2] J. Lee, F. Solat, T. Y. Kim, and H. V. Poor, "Federated learning-empowered mobile network management for 5G and beyond networks: From access to core," *IEEE Commun. Surv. Tut.*, vol. 26, no. 3, pp. 2176–2212, 3rd Quart., 2024.
- [3] J. He, S. Guo, M. Li, and Y. Zhu, "AceFL: Federated learning accelerating in 6G-enabled mobile edge computing networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 3, pp. 1364–1375, May/Jun. 2023.
- [4] C. Benzaid and T. Taleb, "AI-driven zero touch network and service management in 5G and beyond: Challenges and research directions," *IEEE Netw.*, vol. 34, no. 2, pp. 186–194, Mar./Apr. 2020.
- [5] M. Liyanage et al., "A survey on zero touch network and service management (ZSM) for 5G and beyond networks," *J. Netw. Comput. Appl.*, vol. 203, 2022, Art. no. 103362.
- [6] M. El Rajab, L. Yang, and A. Shami, "Zero-touch networks: Towards next-generation network automation," *Comput. Netw.*, vol. 243, 2024, Art. no. 110294.
- [7] G. Hu, X. Xiao, M. Shen, B. Zhang, X. Yan, and Y. Liu, "TCGNN: Packet-grained network traffic classification via graph neural networks," *Eng. Appl. Artif. Intell.*, vol. 123, 2023, Art. no. 106531.
- [8] R. Antonello et al., "Deep packet inspection tools and techniques in commodity platforms: Challenges and trends," *J. Netw. Comput. Appl.*, vol. 35, no. 6, pp. 1863–1878, 2012.
- [9] R. Gudla, S. Vollala, K. Srinivasa, and R. Amin, "A novel approach for classification of tor and non-tor traffic using efficient feature selection methods," *Expert Syst. Appl.*, vol. 249, 2024, Art. no. 123544.
- [10] S. Dong, "Multi class SVM algorithm with active learning for network traffic classification," *Expert Syst. Appl.*, vol. 176, 2021, Art. no. 114885. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0957417421003262>
- [11] J. Dai, X. Xu, H. Gao, and F. Xiao, "CMFTC: Cross modality fusion efficient multitask encrypt traffic classification in IIoT environment," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 6, pp. 3989–4009, Nov./Dec. 2023.
- [12] H. Lu, Y. Dong, Z. Wu, H.-L. Wei, and G. Lu, "New class detection in network traffic classification using confidence information embedded cascade structure," *IEEE Trans. Netw. Sci. Eng.*, vol. 12, no. 3, pp. 1692–1706, May/Jun. 2025.
- [13] Z. Li, P. Wang, and Z. Wang, "FlowGANAnomaly: Flow-based anomaly network intrusion detection with adversarial learning," *Chin. J. Electron.*, vol. 33, no. 1, pp. 1–14, 2024.
- [14] Y.-J. Liu et al., "A survey of integrating generative artificial intelligence and 6G mobile services: Architectures, solutions, technologies and outlooks," *IEEE Trans. Cogn. Commun. Netw.*, vol. 11, no. 3, pp. 1334–1356, Jun. 2025.
- [15] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist.*, vol. 54, 2017, pp. 1273–1282.
- [16] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Trans. Intell. Syst. Technol.*, vol. 10, no. 2, pp. 1–19, 2019.
- [17] W. Y. B. Lim et al., "Federated learning in mobile edge networks: A comprehensive survey," *IEEE Commun. Surv. Tut.*, vol. 22, no. 3, pp. 2031–2063, 3rd Quart., 2020.
- [18] Y.-J. Liu et al., "Trusted clustering based federated learning in edge networks," *IEEE Trans. Mobile Comput.*, vol. 24, no. 10, pp. 9726–9742, Oct. 2025.
- [19] Y. Liu, J. J. Q. Yu, J. Kang, D. Niyato, and S. Zhang, "Privacy-preserving traffic flow prediction: A federated learning approach," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 7751–7763, Aug. 2020.
- [20] E. Bakopoulou, B. Tillman, and A. Markopoulou, "FedPacket: A federated learning approach to mobile packet classification," *IEEE Trans. Mobile Comput.*, vol. 21, no. 10, pp. 3609–3628, Oct. 2022.
- [21] Z. Zhu et al., "MFABA: A more faithful and accelerated boundary-based attribution method for deep neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2024, vol. 38, no. 15, pp. 17228–17236.

- [22] A. Mantelero, "The EU proposal for a general data protection regulation and the roots of the 'right to be forgotten,'" *Comput. Law Secur. Rev.*, vol. 29, no. 3, pp. 229–235, 2013.
- [23] E. L. Harding, J. J. Vanto, R. Clark, L. H. Ji, and S. C. Ainsworth, "Understanding the scope and impact of the california consumer privacy act of 2018," *J. Data Protection Privacy*, vol. 2, no. 3, pp. 234–253, 2019.
- [24] Z. Wang, Z. Li, M. Fu, Y. Ye, and P. Wang, "Network traffic classification based on federated semi-supervised learning," *J. Syst. Archit.*, vol. 149, 2024, Art. no. 103091. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1383762124000286>
- [25] W. Zixuan, M. Cheng, X. Yuhua, L. Zeyi, S. Zhixin, and W. Pan, "Trusted encrypted traffic intrusion detection method based on federated learning and autoencoder," *China Commun.*, vol. 21, no. 8, pp. 211–235, 2024.
- [26] Y. Liu, L. Xu, X. Yuan, C. Wang, and B. Li, "The right to be forgotten in federated learning: An efficient realization with rapid retraining," in *Proc. IEEE INFOCOM 2022-IEEE Conf. Computer Commun.*, 2022, pp. 1749–1758.
- [27] N. Su and B. Li, "Asynchronous federated unlearning," in *Proc. IEEE INFOCOM 2023-IEEE Conf. Comput. Commun.*, 2023, pp. 1–10.
- [28] L. Graves, V. Nagisetty, and V. Ganesh, "Amnesiac machine learning," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, no. 13, pp. 11516–11524.
- [29] J. Wang, S. Guo, X. Xie, and H. Qi, "Federated unlearning via class-discriminative pruning," in *Proc. ACM Web Conf.*, 2022, pp. 622–632.
- [30] Z. Li et al., "Multi-ARCL: Multimodal adaptive relay-based distributed continual learning for encrypted traffic classification," *J. Parallel Distrib. Comput.*, vol. 201, 2025, Art. no. 105083. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0743731525000504>
- [31] S. Fathi-Kazerooni and R. Rojas-Cessa, "Countering machine-learning classification of applications by equalizing network traffic statistics," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 4, pp. 3392–3403, Oct.–Dec. 2021.
- [32] K. Trang and A. H. Nguyen, "A comparative study of machine learning-based approach for network traffic classification," *Knowl. Eng. Data Sci.*, vol. 4, no. 2, pp. 128–137, 2021.
- [33] M. Shen, M. Wei, L. Zhu, and M. Wang, "Classification of encrypted traffic with second-order Markov chains and application attribute bigrams," *IEEE Trans. Inf. Forensics Secur.*, vol. 12, no. 8, pp. 1830–1843, Aug. 2017.
- [34] D. Herrmann, R. Wendolsky, and H. Federrath, "Website fingerprinting: Attacking popular privacy enhancing technologies with the multinomial naive-bayes classifier," in *Proc. ACM Workshop Cloud Comput. Secur.*, 2009, pp. 31–42.
- [35] Y. Wang, Y. Xiang, J. Zhang, W. Zhou, G. Wei, and L. T. Yang, "Internet traffic classification using constrained clustering," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 11, pp. 2932–2943, Nov. 2014.
- [36] Z. Wang et al., "DFE: Deep flow embedding for robust network traffic classification," *IEEE Trans. Netw. Sci. Eng.*, vol. 12, no. 3, pp. 1597–1612, May/Jun. 2025.
- [37] V. G. d. S. Ruffo, L. F. Carvalho, J. Lloret, and M.L. P. Jr, "f-AnoGAN for unsupervised attack detection in SDN environment," *IEEE Trans. Netw. Sci. Eng.*, vol. 12, no. 4, pp. 3271–3285, Jul./Aug. 2025.
- [38] W. Dong, J. Yu, X. Lin, G. Gou, and G. Xiong, "Deep learning and pre-training technology for encrypted traffic classification: A comprehensive review," *Neurocomputing*, vol. 617, 2025, Art. no. 128444.
- [39] X. Wang, Y. Han, V. C. Leung, D. Niyato, X. Yan, and X. Chen, "Convergence of edge computing and deep learning: A comprehensive survey," *IEEE Commun. Surv. Tut.*, vol. 22, no. 2, pp. 869–904, 2nd Quart., 2020.
- [40] S. A. H. Minoofam, A. Bastanfard, and M. R. Keyvanpour, "TRCLA: A transfer learning approach to reduce negative transfer for cellular learning automata," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 5, pp. 2480–2489, May 2023.
- [41] W. Wei, H. Gu, W. Deng, Z. Xiao, and X. Ren, "ABL-TC: A lightweight design for network traffic classification empowered by deep learning," *Neurocomputing*, vol. 489, pp. 333–344, 2022.
- [42] H. Yao, C. Liu, P. Zhang, S. Wu, C. Jiang, and S. Yu, "Identification of encrypted traffic through attention mechanism based long short term memory," *IEEE Trans. Big Data*, vol. 8, no. 1, pp. 241–252, Feb. 2022.
- [43] W. Zheng, C. Gou, L. Yan, and S. Mo, "Learning to classify: A flow-based relation network for encrypted traffic classification," in *Proc. Web Conf.*, 2020, pp. 13–22.
- [44] T. Shapira and Y. Shavitt, "FlowPic: Encrypted internet traffic classification is as easy as image recognition," in *Proc. IEEE INFOCOM 2019-IEEE Conf. Comput. Commun. Workshops*, 2019, pp. 680–687.
- [45] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, "End-to-end encrypted traffic classification with one-dimensional convolution neural networks," in *Proc. IEEE Int. Conf. Intell. Secur. Inform.*, 2017, pp. 43–48.
- [46] M. Lotfollahi, M. Jafari, R. Siavoshani, S. H. Zade, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Comput.*, vol. 24, no. 3, pp. 1999–2012, 2020.
- [47] C. Dong, C. Zhang, Z. Lu, B. Liu, and B. Jiang, "CETAnalytics: Comprehensive effective traffic information analytics for encrypted traffic classification," *Comput. Netw.*, vol. 176, 2020, Art. no. 107258.
- [48] V. Tong, H. A. Tran, S. Souihi, and A. Mellouk, "A novel quic traffic classifier based on convolutional neural networks," in *Proc. IEEE Glob. Commun. Conf.*, 2018, pp. 1–6.
- [49] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-Net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE INFOCOM 2019-IEEE Conf. Comput. Commun.*, 2019, pp. 1171–1179.
- [50] F. Al-Obaidy, S. Momtahan, M. F. Hossain, and F. Mohammadi, "Encrypted traffic classification based ML for identifying different social media applications," in *Proc. IEEE Can. Conf. Elect. Comput. Eng.*, 2019, pp. 1–5.
- [51] G. Xie, Q. Li, and Y. Jiang, "Self-attentive deep learning method for online traffic classification and its interpretability," *Comput. Netw.*, vol. 196, 2021, Art. no. 108267.
- [52] X. Ren, H. Gu, and W. Wei, "Tree-RNN: Tree structural recurrent neural network for network traffic classification," *Expert Syst. with Appl.*, vol. 167, 2021, Art. no. 114363.
- [53] A. M. Sadeghzadeh, S. Shiravi, and R. Jalili, "Adversarial network traffic: Towards evaluating the robustness of deep-learning-based network traffic classification," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 2, pp. 1962–1976, Jun. 2021.
- [54] S. Agrawal et al., "Federated learning for intrusion detection system: Concepts, challenges and future directions," *Comput. Commun.*, vol. 195, pp. 346–361, 2022.
- [55] Y.-J. Liu, S. Qin, Y. Sun, and G. Feng, "Resource consumption for supporting federated learning in wireless networks," *IEEE Trans. Wireless Commun.*, vol. 21, no. 11, pp. 9974–9989, Nov. 2022.
- [56] A. Ariffin, F. Afifi, F. Zaki, H. Hanif, and N. B. Anuar, "Federated learning in network traffic classification: Taxonomy of implementation, application, and impact on sixth-generation wireless networks," *Eng. Appl. Artif. Intell.*, vol. 158, 2025, Art. no. 111471.
- [57] X. He, H. Huang, C. Wang, F. Hu, T. Cai, and Z. Zheng, "A fairness-guaranteed framework for semi-asynchronous federated learning," *IEEE Trans. Netw. Sci. Eng.*, vol. 12, no. 6, pp. 4462–4479, Nov./Dec. 2025.
- [58] O. Alnajjar and A. Barnawi, "Tactile internet of federated things: Toward fine-grained design of FL-based architecture to meet TIIoT demands," *Comput. Netw.*, vol. 231, 2023, Art. no. 109712.
- [59] Y.-J. Liu et al., "Ensemble distillation based adaptive quantization for supporting federated learning in wireless networks," *IEEE Trans. Wireless Commun.*, vol. 22, no. 6, pp. 4013–4027, Jun. 2023.
- [60] N. Wang, X. Li, Z. Guan, and S. Yuan, "FedStream: A federated learning framework on heterogeneous streaming data for next-generation traffic analysis," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 3, pp. 2485–2496, May/Jun. 2024.
- [61] S. Ghosh, "Network traffic analysis based on cybersecurity intrusion detection through an effective automated separate guided attention federated graph neural network," *Appl. Soft Comput.*, vol. 169, 2025, Art. no. 112603.
- [62] Y. Guo and D. Wang, "FEAT: A federated approach for privacy-preserving network traffic classification in heterogeneous environments," *IEEE Internet Things J.*, vol. 10, no. 2, pp. 1274–1285, Jan. 2023.
- [63] Y. Guo, K. Huang, and J. Chen, "WCL: Client selection in federated learning with a combination of model weight divergence and client training loss for internet traffic classification," *Wireless Commun. Mobile Comput.*, vol. 2021, no. 1, 2021, Art. no. 3381998.
- [64] Z. He et al., "Edge device identification based on federated learning and network traffic feature engineering," *IEEE Trans. Cogn. Commun. Netw.*, vol. 8, no. 4, pp. 1898–1909, Dec. 2022.
- [65] X. Huang, Z. Chen, Q. Chen, and J. Zhang, "Federated learning based QoS-aware caching decisions in fog-enabled Internet of Things networks," *Digit. Commun. Netw.*, vol. 9, no. 2, pp. 580–589, 2023.

- [66] W. Jiang, H. Han, Y. Zhang, and J. Mu, "Federated split learning for sequential data in satellite-terrestrial integrated networks," *Inf. Fusion*, vol. 103, 2024, Art. no. 102141.
- [67] M. J. Idriissi et al., "Fed-ANIDS: Federated learning for anomaly-based network intrusion detection systems," *Expert Syst. Appl.*, vol. 234, 2023, Art. no. 121000. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0957417423015026>
- [68] S. Shi, Y. Guo, D. Wang, Y. Zhu, and Z. Han, "Distributionally robust federated learning for network traffic classification with noisy labels," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 6212–6226, May 2024.
- [69] A. Pekar, L. A. Makara, and G. Biczok, "Incremental federated learning for traffic flow classification in heterogeneous data scenarios," *Neural Comput. Appl.*, vol. 36, no. 32, pp. 20401–20424, 2024.
- [70] A. Bremner-Barr, Y. Harchol, D. Hay, and Y. Koral, "Deep packet inspection as a service," in *Proc. 10th ACM Int. Conf. Emerg. Netw. Experiments Technol.*, 2014, pp. 271–282.
- [71] G. Aceto, D. Ciunzo, A. Montieri, and A. Pescapé, "Distiller: Encrypted traffic classification via multimodal multitask deep learning," *J. Netw. Comput. Appl.*, vol. 183, no. 184, 2021, Art. no. 102985. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1084804521000126>
- [72] H. Gu et al., "Unlearning during learning: An efficient federated machine unlearning method," in *Proc. 33rd Int. Joint Conf. Artif. Intell.*, 2024, pp. 1439–1444, doi: [10.24963/ijcai.2024/446](https://doi.org/10.24963/ijcai.2024/446).
- [73] L. Zhang, T. Zhang, H. Zhang, P. Xiong, and W. Zhou, "FedRecovery: Differentially private machine unlearning for federated learning frameworks," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 4732–4746, 2023.
- [74] T. T. Nguyen et al., "A survey of machine unlearning," *ACM Trans. Intell. Syst. Technol.*, vol. 16, no. 5, pp. 1–46, 2025.
- [75] Z. Pan et al., "Feature-based machine unlearning for vertical federated learning in IoT networks," *IEEE Trans. Mobile Comput.*, vol. 24, no. 6, pp. 5031–5044, Jun. 2025.
- [76] J. Fan, K. Wu, Y. Zhou, Z. Zhao, and S. Huang, "Fast model update for IoT traffic anomaly detection with machine unlearning," *IEEE Internet Things J.*, vol. 10, no. 10, pp. 8590–8602, May 2023.
- [77] Z. Liu et al., "A survey on federated unlearning: Challenges, methods, and future directions," *ACM Comput. Surv.*, vol. 57, no. 1, pp. 1–38, 2024.
- [78] T. Che et al., "Fast federated machine unlearning with nonlinear functional theory," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 4241–4268.
- [79] L. Wu, S. Guo, J. Wang, Z. Hong, J. Zhang, and Y. Ding, "Federated unlearning: Guarantee the right of clients to forget," *IEEE Netw.*, vol. 36, no. 5, pp. 129–135, Sep./Oct. 2022.
- [80] C. Wu, S. Zhu, P. Mitra, and W. Wang, "Unlearning backdoor attacks in federated learning," in *Proc. IEEE Conf. Commun. Netw. Secur.*, 2024, pp. 1–9.
- [81] Y. Zhao, P. Wang, H. Qi, J. Huang, Z. Wei, and Q. Zhang, "Federated unlearning with momentum degradation," *IEEE Internet Things J.*, vol. 11, no. 5, pp. 8860–8870, Mar. 2024.
- [82] H. A. Tran, H. T. T. Binh, and A. Mellouk, "Federated learning for network traffic classification: A knowledge consolidation approach," *IEEE Trans. Netw. Sci. Eng.*, vol. 13, pp. 797–814, 2026.
- [83] X. Yin, Y. Zhu, and J. Hu, "A comprehensive survey of privacy-preserving federated learning: A taxonomy, review, and future directions," *ACM Comput. Surv.*, vol. 54, no. 6, pp. 1–36, 2021.
- [84] J. Xu, Z. Wu, C. Wang, and X. Jia, "Machine unlearning: Solutions and challenges," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 8, no. 3, pp. 2150–2168, Jun. 2024.
- [85] S. Liu et al., "Rethinking machine unlearning for large language models," *Nature Mach. Intell.*, vol. 7, pp. 181–194, 2025.
- [86] Z. Li, Z. Zhang, M. Fu, and P. Wang, "A novel network flow feature scaling method based on cloud-edge collaboration," in *Proc. IEEE 22nd Int. Conf. Trust, Secur. Privacy Comput. Commun.*, 2023, pp. 1947–1953.
- [87] W. Wang, S. Jian, Y. Tan, Q. Wu, and C. Huang, "Representation learning-based network intrusion detection system by capturing explicit and implicit feature interactions," *Comput. Secur.*, vol. 112, 2022, Art. no. 102537.
- [88] G. Aceto, D. Ciunzo, A. Montieri, V. Persico, and A. Pescapé, "Mirage: Mobile-app traffic capture and ground-truth creation," in *Proc. 4th Int. Conf. Comput., Commun. Secur.*, 2019, pp. 1–8.
- [89] S. Dadkhah, H. Mahdikhani, P. K. Danso, A. Zohourian, K. A. Truong, and A. A. Ghorbani, "Towards the development of a realistic multidimensional IoT profiling dataset," in *Proc. 19th Annu. Int. Conf. Privacy, Secur. and Trust*, 2022, pp. 1–11.
- [90] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, "End-to-end encrypted traffic classification with one-dimensional convolution neural networks," in *Proc. IEEE Int. Conf. Intell. Secur. Inform.*, 2017, pp. 43–48.



Zeyi Li (Graduate Student Member, IEEE) received the B.S. degree in mathematics from the Suzhou University of Technology, Soochow, China, in 2019, and the M.S. degree in computer science in 2022 from the Nanjing University of Posts and Telecommunications, Nanjing, China, where he is currently working toward the Ph.D. degree in cyberspace security. His research interests include network security, communication network security, anomaly detection and analysis, deep packet inspection, and graph neural networks.



resource allocation.

Yuna Jiang (Member, IEEE) received the Ph.D. degree from the Huazhong University of Science and Technology, Wuhan, China, in 2023. She was a Visiting Ph.D. Student with the School of Computer Science and Engineering, Nanyang Technological University, Singapore. She is currently an Associate Professor with the School of Communications and Information Engineering, Nanjing University of Posts and Telecommunications, Nanjing, China. Her research interests include immersive communication and wireless



Tianshun Wang (Member, IEEE) received the B.Sc. degree in communication engineering from Jilin University, Changchun, China, in 2020, and the Ph.D. degree in computer science from the University of Macau, Macau, China, in 2023. He is currently an Assistant Professor with the School of Communication and Information Engineering, Nanjing University of Posts and Telecommunications, Nanjing, China. His research interests include mobile edge computing, federated learning, and multimodal learning.



and communication network security, network measurements, quality of service, deep packet inspection, software-defined networking, Big Data analytics and applications.

Pan Wang (Member, IEEE) received the B.S. degree in communication engineering and the Ph.D. degree in electrical and computer engineering from the Nanjing University of Posts and Telecommunications, Nanjing, China, in 2001 and 2013, respectively. From 2017 to 2018, he was a Visiting Scholar with the Department of Electrical and Computer Engineering, University of Dayton, Dayton, OH, USA. He is currently a Professor with the School of Modern Posts, Nanjing University of Posts and Telecommunications. His research interests include cyber security



obtained 7 provincial and municipal science and technology progress awards, published and employed more than 20 SCI papers, and obtained 4 authorized invention patents and 3 software copyrights. His research interests include security and applications in cloud computing, BigData, IoT, and AI. From 2009 to 2011, he was a temporary Deputy Director of the Administrative Committee of Wujin National High tech Zone in Jiangsu Province (assisting in the work of scientific and technological talents, and won the title of preferred communist party member during this period).

Yimu Ji was born in 1978. He received the Ph.D. degree from the Nanjing University of Posts and Telecommunications (NJUPT), Nanjing, China, in 2007. From September 2012 to December 2012, he was sent to the University of Virginia for a UVA study visit, and from June 2016 to December 2016, he was sent to Purdue University for a study visit by the Jiangsu Provincial Government. He is currently a Professor with the School of Computer Science, NJUPT. He presided more than and participated in many national, provincial and ministerial projects,